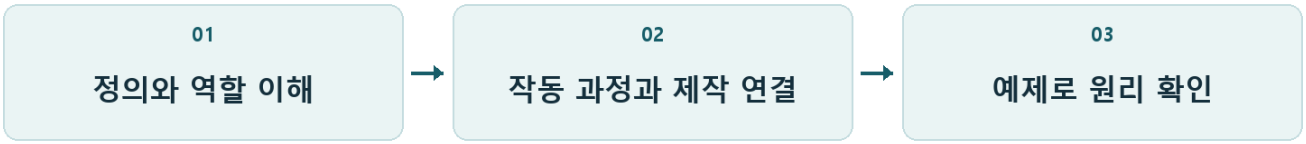
언리얼 엔진 5 렌더링 원리와 최적화

게임 배경 아티스트를 위한 기술 학습 시리즈

이 문서가 설명하는 것

모델과 재질, 빛이 화면으로 바뀌는 전체 구조에서 각 기술이 무엇이고 어떤 일을 하는지 배웁니다. 먼저 개념과 작동 과정을 이해하고, 왜 배경 제작에 필요한지 연결한 뒤 예제로 일부 원리를 확인합니다.



학습의 순서

렌더링 파이프라인의 전체 구조에서 시작해 재질과 빛, 에셋 처리, 시간적 화질, 성능 분석으로 이어집니다. 각 챕터는 정의·역할·입력과 출력·작동 과정을 먼저 설명합니다. 사례는 주제의 한 적용이고 실습은 본문에서 배운 관계를 확인하는 수단입니다.

실습과 자료의 상태

UE 5.8.4 데스크톱 프로젝트를 기준으로 합니다. CH01에는 직접 제작한 쇼윈도와 실제 캡처가 있습니다. CH02부터 CH26은 기본 레벨과 제작·검증 절차이며, 이 개정에서 새 예제 제작이나 성능 실측을 완료한 것은 아닙니다.

이미지의 구분

엔진 캡처, 직접 제작 개념도, 외부 렌더·진단 화면을 캡션에서 구분합니다. 외부 예시의 버전·수치를 현재 프로젝트의 결과로 옮기지 않습니다.

목차

렌더링 기초

01 렌더링 파이프라인 · 4
02 Forward와 Deferred · 10
03 GBuffer와 표면 정보 · 12
04 DBuffer와 데칼 · 14
05 Depth와 Early Z · 16
06 Overdraw와 화면 겹침 · 18

머티리얼과 빛

07 PBR과 재질 입력 · 20
08 BRDF와 반사 분포 · 22
09 Fresnel과 시선 각도 · 24
10 셰이더와 머티리얼 · 26
11 Substrate와 물질 구성 · 28
12 Translucency와 합성 · 30
13 SSS와 표면 아래 산란 · 32

지오메트리와 텍스처

14 Draw Call과 제출 작업 · 34
15 Instancing과 반복 배치 · 36
16 LOD와 화면 크기 · 38
17 Nanite와 가상화 지오메트리 · 40
18 Culling과 가시성 · 42
19 Mipmap과 텍스처 샘플링 · 44
20 Texture Streaming · 46

라이팅과 화면 품질

21 Lumen과 간접광 · 48
22 Virtual Shadow Maps · 50
23 TAA와 프레임 히스토리 · 52
24 TSR과 시간적 재구성 · 54

실전 성능 분석

25 CPU와 GPU 병목 · 56
26 GPU Profiler와 개선 검증 · 58

이 문서를 읽는 방법 · 3

종합 실습 · 60

용어집 · 61

심화와 적용 조건 · 67

출처 목록 · 68

이 문서를 읽는 방법

첫째 개념과 역할부터 읽습니다

먼저 이것이 무엇인지, 렌더링의 어디에서 어떤 일을 맡는지 읽습니다. 주제 전체를 한 예제의 문제로 축소하지 않고 기본 구조를 이해합니다.

둘째 입력과 출력을 따라갑니다

무엇을 받아 어떤 계산을 하고 무엇을 다음 단계에 넘기는지 확인합니다. 단계 이름을 외우기보다 데이터가 화면 결과로 바뀌는 연결을 읽습니다.

셋째 제작에 필요한 이유를 읽습니다

형상·텍스처·재질·조명·배치 중 어떤 선택에 관여하는지 확인합니다. 특정 실습을 끝내는 것이 아니라 다른 장면에서도 올바른 수정 대상을 선택하는 것이 목적입니다.

넷째 예제로 일부 원리를 확인합니다

본문을 읽은 뒤 그림의 관찰 지점과 예상 결과를 봅니다. 예제는 전체 기술을 증명하는 실험이 아닙니다. 무엇을 확인하는 작은 비교인지 목표를 먼저 정합니다.

성능 비교의 공통 조건

기기·엔진·그래픽 API·출력 크기·카메라·노출·품질을 기록합니다. 셰이더 준비와 로딩을 끝낸 뒤 같은 경로를 반복합니다. FPS와 함께 작업 시간인 ms를 기록하고, 필요한 외관이 유지되는지 봅니다.

본문 밖으로 옮긴 내용

정의는 처음 필요한 자리에서 설명하고, 용어집은 재확인에 사용합니다. 수식·순열·모델별 입력·저장 형식·도구 버전은 본문 결론을 이해한 뒤 심화와 공식 자료에서 확인합니다.

01 렌더링 파이프라인

렌더링 파이프라인이란

렌더링 파이프라인은 모델·재질·조명·카메라로 이루어진 장면 데이터를 계산해, 화면에 표시할 픽셀의 색으로 만드는 일련의 처리 과정입니다. 파이프라인이라는 말은 한 작업의 결과를 다음 작업의 입력으로 넘기는 구조를 뜻합니다.

장면 데이터가 이미지가 되는 구조

3D 장면은 그 자체로 사진이 아닙니다. 물체가 어디에 있고, 화면의 어디를 덮으며, 어떤 표면에 어떤 빛이 도달하는지 계산해야 이미지가 됩니다. 파이프라인은 이 일을 여러 단계로 나누고 필요한 중간 정보를 전달하는 렌더링의 전체 구조입니다.

입력과 출력

입력은 메시의 정점·배치, 텍스처와 재질, 광원, 카메라, 렌더링 설정입니다. 출력은 표시할 2D 이미지입니다. 중간에는 화면 위치, 깊이, 표면 특성, 조명 결과 같은 정보가 생깁니다.

처리 단계	입력으로 받는 정보	다음에 전달하는 결과
그릴 작업 준비	메시·배치·재질·카메라·장면 상태	처리할 후보와 그리기 명령
형태를 화면에 대응	정점 위치와 변환·카메라	화면의 삼각형·샘플과 보간 속성
깊이와 표면 계산	표면 후보·텍스처·재질	앞뒤 관계와 색·거칠기·법선 등
조명과 그림자 반영	표면 속성·광원·가림과 간접광	조명된 표면의 색
합성과 후처리	표면 색·반투명·노출과 효과	표시할 최종 이미지

대표적인 래스터 렌더링의 의존 관계를 정리한 표입니다. 그림자 등 별도 작업과 병렬 실행을 생략했으며 실제 엔진의 고정 실행 순서표는 아닙니다.

Epic 공식 구조 설명 [62]

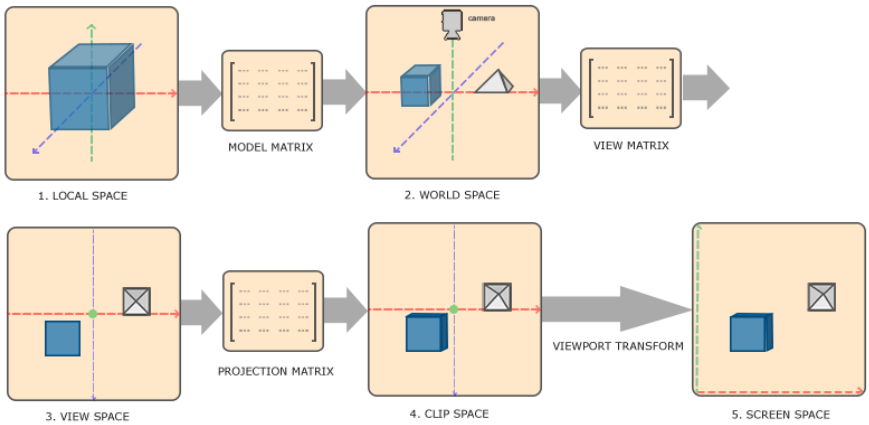
왜 배경 아티스트가 알아야 할까

내가 바꾼 형상·텍스처·재질·조명이 화면의 어느 계산에 영향을 주는지 알기 위해서입니다. 전체 구조를 이해한 뒤 각 단계와 제작의 연결을 이어서 설명합니다.

모델의 위치가 화면 위치가 되는 과정

형태를 이루는 정보

지오메트리는 물체의 위치와 모양을 정하는 기하 정보입니다. 메시의 표면은 삼각형으로 표현할 수 있고, 정점은 그 꼭짓점입니다. 정점 셰이더는 GPU에서 이 위치 등을 계산하는 프로그램입니다.



외부 참고 그림 · 모델의 로컬 공간부터 월드·카메라·클립·화면 공간까지 이어지는 좌표 변환입니다.

1→2에서는 큐브를 장면에 배치하고, 2→3에서는 카메라를 기준으로 장면을 해석합니다. 4→5에서는 투영한 결과가 화면 영역에 대응합니다. UE와 OpenGL의 축·깊이 규약은 같지 않지만 좌표 기준을 바꾸는 목적은 같습니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [37]

원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

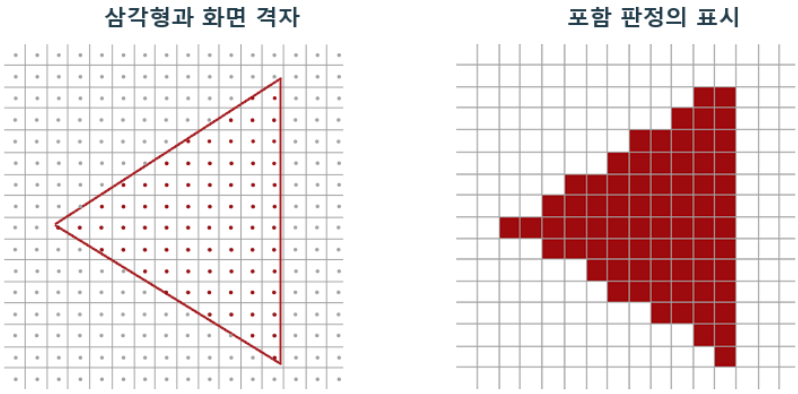
좌표가 달라지는 이유

모델 내부의 꼭짓점 위치에 배치의 이동·회전·크기를 반영하면 월드 위치가 됩니다. 이후 카메라를 기준으로 표현하고 투영해 화면 위치를 구합니다. 예를 들어 회전·확대 없이 원점을 X=10에 놓으면 로컬 X=1인 점의 월드 X는 11입니다.

아트 작업에서의 구분

위치 변환은 모델을 어디에 보이게 할지 정합니다. UV는 표면의 어느 텍스처 위치를 읽을지, 법선은 표면이 어느 방향을 향하는지 정합니다. 위치·무늬·반사의 방향을 서로 다른 정보로 구분합니다.

삼각형 내부의 화면 값을 만드는 과정



외부 비교 그림 · 삼각형과 화면 격자, 그리고 포함 판정을 표시한 결과를 나란히 배치했습니다.

왼쪽의 매끈한 삼각형 경계와 오른쪽의 격자에 맞춘 계단형 경계를 비교합니다. 이 그림은 래스터화와 앨리어싱의 관계를 설명하며 TAA 또는 TSR의 전후 결과가 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [38]
원본 전체 유지 · 비율 유지 · CC BY 4.0
이용 조건

래스터화

래스터화는 화면으로 옮긴 삼각형이 어느 샘플 위치를 덮는지 판정하는 과정입니다. 샘플은 검사하는 화면 위치입니다. 삼각형의 격자 포함 여부와 가려짐 검사를 거쳐 최종 화면에 기여할 후보를 찾습니다.

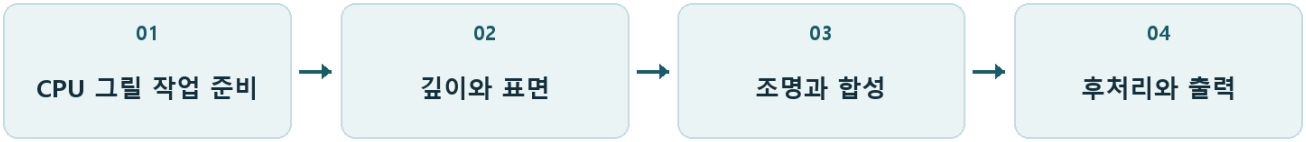
보간

보간은 알려진 값 사이의 중간값을 구하는 것입니다. 꼭짓점에 있는 UV 등을 이용해 삼각형 내부 위치의 값을 계산하므로, 가운데에도 텍스처를 연속적으로 붙일 수 있습니다. 원근 투영에서는 이에 맞는 보정이 적용됩니다.

왜 형상과 텍스처를 구분할까

텍스처는 삼각형 안의 무늬를 세밀하게 만들 수 있지만 바깥 실루엣의 곡선을 직접 늘리지는 않습니다. 표면 무늬가 거친지, 형상 윤곽이 각진지에 따라 수정할 데이터를 고릅니다.

표면 정보에서 최종 이미지까지



직접 제작 개념도. 작업 의존 관계이며 GPU 전체 실행 순서표는 아닙니다.

그릴 작업의 준비

그릴 작업을 준비합니다. CPU는 장면 상태와 카메라를 바탕으로 후보를 정리하고 명령을 준비합니다. 렌더 스레드는 이 준비를 맡는 CPU 실행 흐름이며, RHI는 명령을 DirectX·Vulkan 같은 그래픽 API에 연결하는 계층입니다. GPU는 제출된 일을 실행합니다.

깊이와 표면 데이터

버퍼는 계산 중 쓸 데이터를 보관하는 메모리 공간입니다. 깊이는 같은 화면 위치의 앞뒤를 판정하는 데 쓰입니다. 일반적인 Deferred 경로의 Base Pass는 재질로 표면 특성을 구해 GBuffer에 기록하고, 조명 작업이 이를 읽습니다. 표면을 그릴 때 조명도 평가하는 Forward와는 분리 방식이 다릅니다.

조명은 표면 정보에 빛을 결합합니다

기본 색·거칠기·법선은 물체의 최종 사진색이 아닙니다. 광원의 방향과 밝기, 환경의 간접광·반사와 그림자를 결합해야 조명된 색이 됩니다. 그림자 작업은 광원 관점의 가림 정보를 만들어 조명에 전달하므로 메인 카메라에 보이는 메시를 그리는 일과 별개로 비용이 생길 수 있습니다.

합성과 후처리가 표시할 이미지를 만듭니다

유리·연기처럼 뒤가 비치는 표면은 그 뒤의 색과 합성해야 합니다. 후처리에는 장면 밝기의 노출 조절, 넓은 밝기 범위를 표시 범위로 옮기는 톤매핑, 밝은 부분을 퍼뜨리는 Bloom 등이 있습니다. 실제 적용 시점과 구성은 효과와 렌더 경로에 따릅니다.

렌더 패스는 작업을 나눈 단위입니다

패스는 특정 목적의 렌더링 작업 단위입니다. 같은 메시도 깊이·기본 표면·그림자 패스에 참여할 수 있습니다. 단계 그림은 데이터의 역할을 이해하기 위한 것이며 CPU와 GPU가 모든 단계를 하나씩 직렬로 기다린다는 뜻은 아닙니다.

배경 제작의 선택은 어디에 영향을 줄까

윤곽과 굽힘

정점·삼각형·변형을 바꿉니다. 큰 곡선의 실루엣은 텍스처 해상도만 올려 해결하지 못합니다.

표면 무늬와 재료

텍스처·UV·색·거칠기·법선을 바꿉니다. 무늬가 흐린 문제와 형상이 각진 문제를 구분합니다.

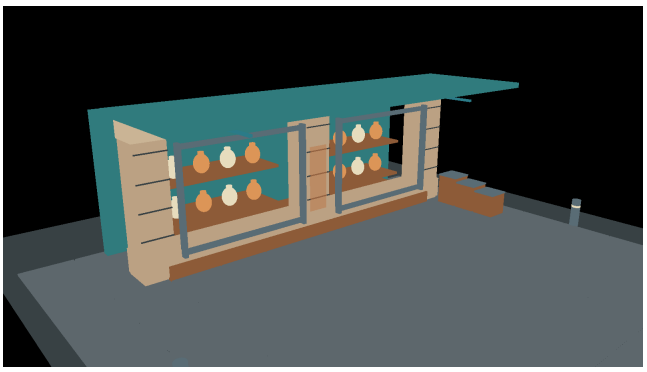
밝기와 반사 환경

광원·그림자·간접광과 환경 반사를 바꿉니다. 표면 입력이 같아도 들어오는 빛이 바뀌면 외관이 달라집니다.

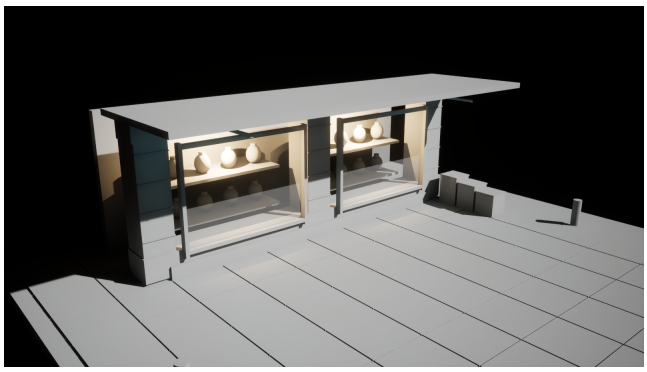
번짐과 최종 색

발광·노출·톤매핑·Bloom 등을 봅니다. 밝은 번짐이 과한 것을 기본 표면색만 낮춰 고치면 다른 조명에서 재료가 어두워질 수 있습니다.

Base Color



Lighting Only



직접 제작·같은 카메라의 엔진 캡처. 왼쪽은 기본 표면색, 오른쪽은 조명 기여를 읽는 진단입니다. 두 화면을 더하면 Lit가 된다는 분해식은 아닙니다.

성능에서도 줄이는 일이 다릅니다

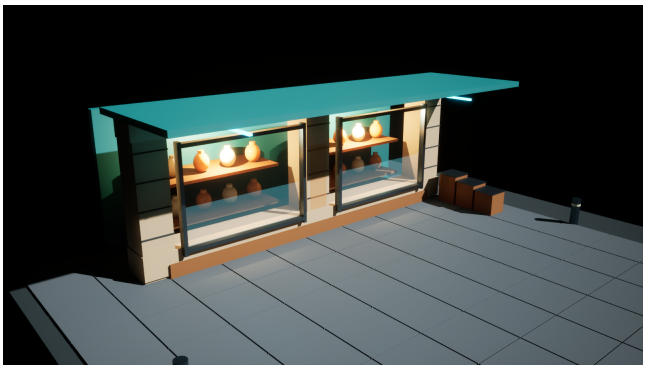
한 메시도 깊이·카메라 표면·그림자 등 여러 작업에 참여할 수 있습니다. 삼각형 수, 그리는 화면 면적, 재질 계산, 광원과 중간 데이터의 읽기·쓰기가 서로 다른 비용을 만듭니다. 무엇을 줄일지는 실제로 느린 작업을 확인한 뒤 정합니다.

예상하고 확인하는 첫 실습

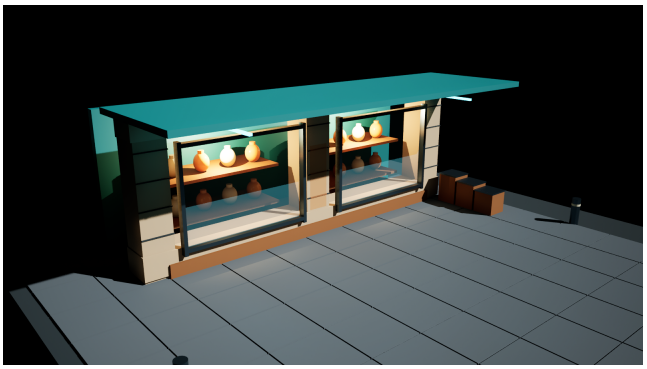
예제 목표

후처리의 변화와 기본 표면 입력을 구분합니다.

Bloom 적용



Bloom 해제



직접 제작·동일 구도의 엔진 비교. 밝은 바의 주변 번짐을 봅니다. 캡처 중의 시간은 성능 실측으로 사용하지 않습니다.

실습 절차

CH01 쇼윈도에서 같은 카메라를 유지합니다. Lit와 Base Color를 먼저 비교한 뒤 Bloom만 끄고 Lit와 Base Color를 다시 봅니다. 노출과 재질 값은 그대로 둡니다.

예상 결과와 이유

Bloom을 끄면 밝은 바 주변의 번짐이 줄어들지만, 기본 표면색은 같은 상태로 남을 것입니다. 이는 화면 뒤쪽 단계의 변화가 앞쪽 표면 입력을 바꾸지 않는다는 예상입니다.

결과가 다를 때

번짐과 표면색이 함께 바뀌면 Bloom 외의 설정이나 노출이 달라졌는지 먼저 확인합니다. 진단 화면이 달라진 이유를 찾는 것이 이번 실습의 목적이며, 이 비교만으로 성능 향상까지 판단하지 않습니다.

이 챕터에서 기억할 것

렌더링 파이프라인은 장면 데이터를 이미지로 바꾸는 전체 계산 구조이며, 아트 데이터의 변경이 어느 계산과 화면 결과를 바꾸는지 연결해 이해하는 기준입니다.

레벨 /Game/RenderingLab/Chapters/Part01_RenderingBasics/CH01_RenderingPipeline/Maps/L_CH01_RenderingPipeline

02 Forward와 Deferred

무엇인가

Forward와 Deferred는 표면의 재질 정보와 조명을 결합하는 방식을 나누는 대표적인 렌더링 구조입니다. 핵심 차이는 표면을 그릴 때 조명까지 평가하는지, 표면 정보를 먼저 저장하고 뒤에서 조명을 평가하는지입니다.

렌더링에서 맡는 역할

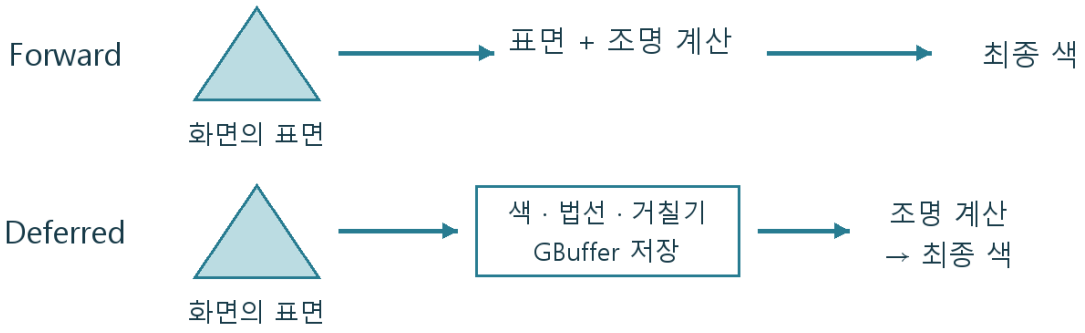
둘 다 장면을 조명된 이미지로 만들지만 작업을 나누는 위치가 다릅니다. 그래서 필요한 중간 버퍼, 기능 지원과 비용의 분포도 달라집니다. 프로젝트의 렌더러 선택은 재질·조명·AA 제작 조건을 정하는 상위 결정입니다.

입력과 출력

공통 입력은 보이는 지오메트리, 재질과 광원입니다. Forward는 표면 계산에서 조명된 색을 만들고, Deferred는 표면 데이터인 GBuffer를 먼저 만든 뒤 조명된 색을 만듭니다.

작동 과정

Forward에서는 표면을 그리는 셰이더가 영향을 주는 광원과 재질 반응을 함께 계산합니다. UE는 광원을 선별하는 구조를 사용하므로 모든 표면이 모든 광원을 무조건 계산하는 방식은 아닙니다. Deferred에서는 먼저 보이는 표면의 색·법선·거칠기 등을 기록합니다. 다음 조명 작업이 이 값을 읽어 해당 화면 위치의 빛 반응을 계산합니다. 표면 정보와 조명 평가가 분리됩니다. 이후 반투명과 후처리를 포함한 화면 구성이 이어집니다. 선택한 렌더러에 따라 쓸 수 있는 기능과 구현 경로가 달라지므로, 같은 기능이 켜졌다는 이름만으로 동일한 결과라고 가정하지 않습니다.



차이의 중심은 조명 전에 표면 데이터를 따로 저장하고 읽는가입니다.

직접 제작 개념도. 표면과 조명을 결합하는 시점 및 중간 저장의 차이입니다. 실제 프레임의 모든 패스를 나열한 도식은 아닙니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

VR의 경계 품질, 필요한 반사·조명 기능, 플랫폼의 제약을 먼저 정할 때 사용합니다. 에셋을 만든 뒤 빠르다는 말만 듣고 렌더러를 바꾸면 기존 재질과 조명 설계를 다시 검토해야 할 수 있습니다.

성능과 제작 비용

Deferred는 화면 크기의 중간 표면 데이터를 쓰고 읽는 부담이 있습니다. Forward는 표면 작업에 조명 계산이 포함됩니다. 광원·그림자·재질·해상도와 기능 조건에 따라 총 비용을 비교해야 합니다.

이 예제의 목표

같은 장면에서 표면 처리와 조명 작업이 어디에 배치되는지 확인합니다.

실습 절차

프로젝트 복사본에서 Forward Shading을 바꾸고 재시작합니다. 공통 지원하는 불투명 재질과 그림자 없는 이동형 라이트만 둔 작은 장면을 준비합니다. 해상도와 지원 기능을 맞춘 뒤 광원 수를 한 단계씩 늘려 패스 구성을 비교합니다.

예상 결과와 이유

Deferred에서는 표면을 기록한 뒤 별도 조명 작업이 이어지고, Forward에서는 표면을 그리는 작업에 조명 계산이 포함되는 구조를 확인할 수 있을 것입니다.

결과를 어떻게 해석할까

패스 이름 하나의 시간만 직접 맞대지 않습니다. 계산이 서로 다른 작업으로 나뉘므로 전체 GPU 시간과 화면 결과를 함께 기록합니다. 설정 변경 후 기능이 빠졌다면 동일한 화질의 성능 비교로 사용하지 않습니다.

이 챕터에서 기억할 것

두 방식의 차이는 표면과 조명을 결합하는 위치입니다. 필요한 표현을 먼저 정하고 그 조건을 만족하는 경로의 전체 비용을 비교합니다.

이해한 뒤 확인할 조건

이 장은 데스크톱 렌더링의 기본 구조를 설명합니다. 모바일 경로와 플랫폼별 기능 지원표는 공식 문서에서 별도로 확인합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[01] Rendering Settings

[04] Forward Shading Renderer

03 GBuffer와 표면 정보

무엇인가

GBuffer는 Deferred 조명이 사용할 표면 특성을 화면 위치별로 저장하는 버퍼 묶음입니다. 버퍼는 계산에 쓸 데이터를 보관하는 메모리이며, GBuffer는 최종 사진이 아닌 표면 정보의 중간 저장소입니다.

렌더링에서 맡는 역할

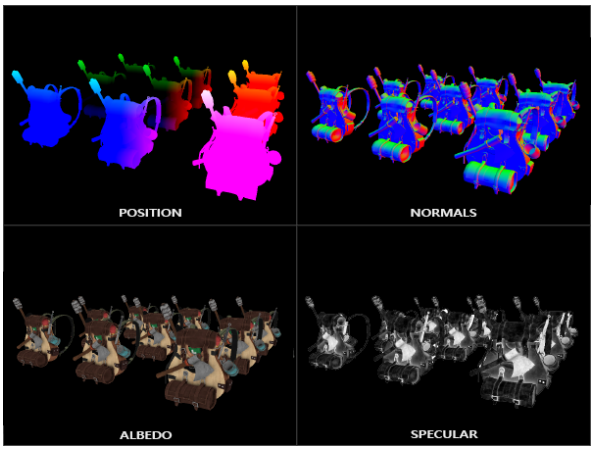
표면의 기본 색과 방향·거칠기를 조명 계산과 분리합니다. 이 데이터를 기록한 뒤 여러 조명과 후속 계산이 재사용할 수 있습니다. 재질 입력과 조명 결과를 따로 진단할 수 있는 이유도 여기에 있습니다.

입력과 출력

입력은 보이는 표면에서 재질이 계산한 속성입니다. 출력은 해당 화면 위치의 표면 데이터이며, 이후 조명은 깊이·카메라·광원 정보와 함께 이를 읽어 조명된 색을 만듭니다.

작동 과정

표면을 그리는 Base Pass에서 텍스처와 재질 입력을 평가합니다. Base Color는 기본 표면색, Roughness는 반사 분포의 거칠기, 법선은 표면 방향을 나타냅니다. 계산한 속성을 정해진 형식으로 저장합니다. Scene Depth는 별도의 깊이 정보입니다. 모든 속성이 각각 독립된 이미지 하나에 저장되는 것은 아니며, 채널 배치는 렌더러와 저장 형식에 따라 달라집니다. 조명 단계가 저장한 값을 읽고 빛의 방향·세기와 표면 반응을 결합합니다. 같은 표면 데이터도 조명이 바뀌면 최종 색이 달라집니다. Buffer Visualization은 일부 정보를 관찰용으로 표시합니다.



외부 시각화 이미지 · 같은 장면의 위치·법선·Albedo·Specular를 분리해 표시한 GBuffer 예시입니다.

왼쪽 위는 위치에 따라, 오른쪽 위는 표면 방향에 따라 색이 달라집니다. 아래쪽은 재질의 색과 반사 입력입니다. 진단용 색을 최종 외관이나 UE 채널 구성으로 읽지 않습니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [39]
원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

벽의 번들거림은 Roughness, 반사의 방향 오류는 World Normal, 고정된 색 얼룩은 Base Color부터 확인할 수 있습니다. 진단이 의도와 다르면 재질 입력을, 입력이 맞으면 조명과 반사 환경을 조사합니다.

성능과 제작 비용

저장할 화면 픽셀 수와 픽셀당 데이터가 메모리 사용과 대역폭을 좌우합니다. 머티리얼 노드 수가 많아졌다는 사실만으로 GBuffer의 바이트 수가 같은 비율로 늘지는 않습니다.

이 예제의 목표

거칠기 변경과 광원 변경이 서로 다른 진단 화면에 나타나는지 확인합니다.

실습 절차

같은 벽의 Material Instance 두 개를 만들고 Roughness만 0.2와 0.8로 둡니다. Material Instance는 원본 재질의 구조를 공유하며 설정값을 바꾸는 변형입니다. 같은 구도에서 Roughness, Base Color, Lit 순서로 비교합니다.

예상 결과와 이유

Roughness 진단은 달라지고 Base Color는 유지되며, Lit에서는 반사의 퍼짐이 달라질 것입니다. 다음으로 라이트 위치만 바꾸면 Lit는 달라져도 두 표면 입력은 유지될 것입니다.

결과를 어떻게 해석할까

첫 비교는 재질 입력이 결과에 미치는 영향을, 두 번째 비교는 같은 입력에 조명이 미치는 영향을 보여 줍니다. 진단 표시색을 실제 저장 바이트 배열로 해석할 필요는 없습니다.

이 챕터에서 기억할 것

GBuffer는 조명이 읽는 표면 데이터입니다. 표면 자체의 오류와 빛을 계산한 뒤의 결과를 구분하는 데 사용합니다.

이해한 뒤 확인할 조건

GBuffer의 채널과 저장량은 렌더러·플랫폼·Substrate 설정에 따라 달라집니다. 해상도와 메모리 비용의 관계는 이 실험과 별개의 성능 확인 항목입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[02] Render Dependency Graph

[05] Programming with Substrate GBuffer Formats

04 DBuffer와 데칼

무엇인가

DBuffer는 데칼이 바꾸는 표면의 색·법선·거칠기 정보를 기본 표면 계산 전에 준비하는 버퍼입니다. 데칼은 오염·손상·표식을 다른 표면에 투영하는 표현이고, DBuffer는 그 변경을 전달하는 중간 데이터입니다.

렌더링에서 맡는 역할

데칼 변경을 벽의 기본 재질 계산에 반영해 이후 조명에 참여하게 합니다. 화면 위에 완성된 그림을 붙이는 것과 달리, 빛을 받을 표면 특성을 수정하는 역할입니다.

입력과 출력

입력은 데칼 재질, 투영 박스, 깊이 정보와 수신 조건입니다. 출력은 수신 표면에 적용할 변경 데이터이며 Base Pass가 이를 읽어 기본 표면과 결합합니다.

작동 과정

데칼의 투영 박스로 적용할 공간을 정합니다. 엔진은 깊이로 찾은 표면 위치가 이 범위 안에 있는지 판단하고 데칼의 텍스처 위치를 계산합니다. 데칼 재질의 변경값을 DBuffer에 준비합니다. 겹치는 데칼은 해당 경로의 합성 규칙에 따라 처리됩니다. 어떤 표면이 받는지는 수신 설정에도 달려 있습니다. 수신 재질의 Base Pass가 데칼 정보를 읽어 색·거칠기·방향 등에 반영합니다. 조명은 이렇게 바뀐 표면에 반응하므로 젖은 자국과 긁힘도 빛의 방향에 따라 다르게 보입니다.



공식 비교 이미지 · 같은 돌벽에 데칼을 적용하기 전과 후입니다. UE 5.5 문서의 사례입니다.

반복되는 벽돌 위에 생긴 손상과 표면 변화가 어디까지 이어지는지 봅니다. 데칼 수나 성능의 통제 실험은 아니므로 투영 면적·겹침의 비용은 실습에서 따로 측정합니다.

© Epic Games · 공식 문서 · 출처 [46]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 원저작권과 원문 출처 표시는 유지합니다

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

모듈 벽의 반복을 깨고 도로 표식이나 얼룩을 추가할 때 필요합니다. 원치 않는 소품에 묻으면 텍스처색보다 투영 범위와 Receives Decals·Decal Response를 먼저 봅니다.

성능과 제작 비용

보이는 화면 면적과 중첩, 깊이 준비, 수신 재질에서 읽는 작업이 비용을 만듭니다. 이미 필요한 깊이 작업 전체를 데칼 하나의 추가 비용으로 계산하지 않습니다.

이 예제의 목표

벽의 얼룩을 유지하면서 박스 깊이만 줄여 잘못 묻는 소품을 제외합니다.

실습 절차

벽 앞에 작은 소품을 놓고 하나의 얼룩 데칼을 투영합니다. 먼저 큰 박스에서 잘못 묻는 표면을 찾고, 데칼의 전면 크기는 유지한 채 박스 깊이만 줄입니다.

예상 결과와 이유

벽의 얼룩은 남고 박스 밖으로 빠진 소품의 얼룩은 사라질 것입니다. 이후 라이트 방향만 바꾸면 얼룩을 포함한 표면이 빛에 함께 반응할 것입니다.

결과를 어떻게 해석할까

소품에 계속 묻으면 실제 박스 위치와 수신 설정을 확인합니다. 색만 비교하지 말고 거칠기나 법선 변경도 확인해야 물기·스크래치 데칼의 효과를 놓치지 않습니다.

이 챕터에서 기억할 것

DBuffer는 데칼의 표면 변경을 기본 재질에 전달합니다. 투영 공간과 수신 조건을 알아야 원하는 표면만 바꿀 수 있습니다.

이해한 뒤 확인할 조건

Receives Decals는 물체의 수신 여부, Decal Response는 재질이 받을 변경을 정합니다. 다른 데칼 경로와 적용 순서는 공식 자료의 심화 항목입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[05] Programming with Substrate GBuffer Formats

[06] Decal Materials

05 Depth와 Early Z

무엇인가

Depth Buffer는 화면 샘플의 앞뒤 관계를 판정하는 깊이 데이터를 보관합니다. Early Z는 그 깊이 판정을 재질 계산보다 일찍 적용해 가려진 후보의 계산을 생략하는 처리입니다.

렌더링에서 맡는 역할

깊이 검사는 불투명 표면의 올바른 가림을 만들고, 조기 깊이 검사는 불필요한 픽셀 계산을 줄일 기회를 줍니다. Depth Prepass는 깊이를 미리 준비하는 작업으로 Early Z 자체와는 구분됩니다.

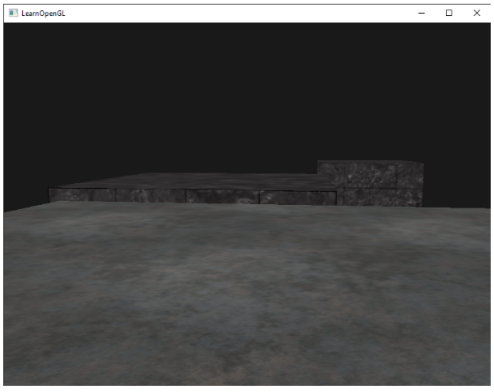
입력과 출력

입력은 투영된 표면의 깊이와 이미 기록된 깊이입니다. 비교 결과에 따라 후보가 남거나 탈락하며, 허용되는 조건에서는 비싼 픽셀 셰이더 실행을 줄일 수 있습니다.

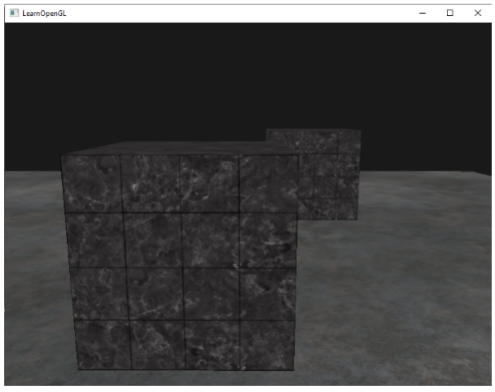
작동 과정

표면을 화면에 투영하면 위치별 깊이도 얻습니다. 깊이 검사로 같은 화면 위치의 앞뒤를 비교하고 설정한 규칙에 맞는 표면을 남깁니다. Prepass를 사용하는 경로에서는 먼저 깊이를 그립니다. 이후 표면 계산이 준비된 깊이를 이용할 수 있습니다. 깊이 준비 없이도 Early Z가 가능한 조건이 있으므로 두 용어를 같은 뜻으로 쓰지 않습니다. 일찍 탈락시킬 수 있는 후보는 재질 계산을 건너뛰게 할 수 있습니다. 재질이 깊이나 가시성 결정에 영향을 주는 경우에는 적용 조건이 달라집니다.

깊이 항상 통과



앞쪽 깊이로 검사



외부 비교 이미지 · 왼쪽은 깊이 비교를 항상 통과, 오른쪽은 앞쪽 깊이를 선택한 OpenGL 예시입니다.

왼쪽에서 바닥이 큐브를 잘못 덮는 부분과 오른쪽의 올바른 가림을 비교합니다. 깊이 검사의 필요성을 보여 주며, Early Z로 몇 ms를 절약했는지 입증하는 이미지는 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [47]
동일 범위의 원본을 나란히 배치 · 표제 추가 · CC BY 4.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

밀집 실내에서 벽이 뒤의 소품을 가리는 효과, 식생의 빈 영역 처리와 가림의 차이를 이해할 때 필요합니다. 최종 화면에 안 보인다는 사실만으로 모든 작업이 사라졌다고 생각하지 않습니다.

성능과 제작 비용

Prepass는 지오메트리를 다시 처리하는 선행 비용이 있습니다. 줄인 표면 계산이 이 비용보다 큰지 봐야 하며, 깊이 표시가 올바르다는 것과 성능 개선은 별도로 확인합니다.

이 예제의 목표

가림 범위가 깊이 표시와 뒤쪽 재질 처리 시간에 미치는 영향을 구분합니다.

실습 절차

비교용 전통 불투명 메시의 뒤 패널에 계산이 많은 재질을 둡니다. 카메라를 고정하고 앞의 단순한 가림판으로 패널을 절반, 대부분 가리는 두 상태를 만듭니다. Prepass 설정은 유지하고 Scene Depth와 Base Pass 시간을 봅니다.

예상 결과와 이유

Scene Depth에는 가림판이 앞면으로 보일 것입니다. 조기 깊이 제거가 유효하고 뒤 패널의 픽셀 계산 비중이 크다면, 더 많이 가린 상태에서 해당 계산 시간이 줄 것으로 예상합니다.

결과를 어떻게 해석할까

시간이 그대로면 뒤 표면이 원래 싸거나, 다른 일이 지배적이거나, 조기 제거 조건이 다를 수 있습니다. 깊이 표시가 맞는다는 것만으로 최적화 효과를 입증하지 않습니다.

이 챕터에서 기억할 것

깊이는 앞뒤를 정하고 Early Z는 가려진 계산을 줄일 기회를 만듭니다. 깊이를 미리 그리는 비용과 절감 효과를 함께 판단합니다.

이해한 뒤 확인할 조건

Masked, 깊이를 바꾸는 Pixel Depth Offset, 반투명은 조건이 다릅니다. 먼저 불투명 재질에서 기본 관계를 이해한 뒤 별도로 비교합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[02] Render Dependency Graph

[03] Introduction to Performance Profiling and Configuration

[06] Decal Materials

[30] Visibility and Occlusion Culling

06 Overdraw와 화면 겹침

무엇인가

Overdraw는 화면의 같은 영역에 여러 표면 후보가 그려져 반복 처리되는 현상입니다. 앞뒤 표면의 겹침, 마스크 검사와 반투명 층 합성 등에서 생길 수 있으며 반복 처리 모두가 불필요한 것은 아닙니다.

렌더링에서 맡는 역할

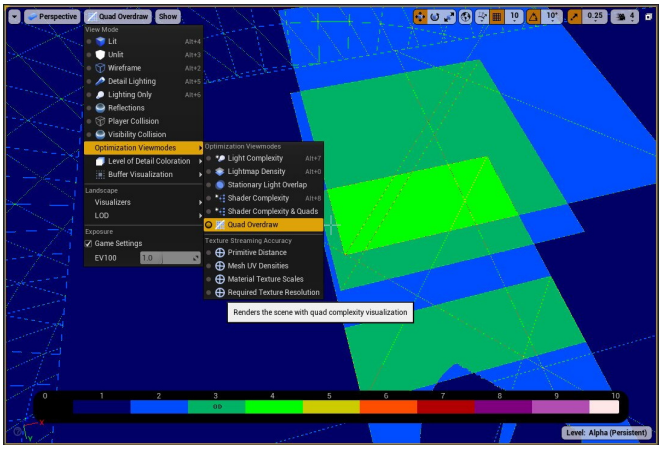
삼각형 개수만으로 설명되지 않는 화면 단위의 작업량을 이해하는 개념입니다. 깊이로 일찍 제거할 수 없는 표면이나 여러 층이 필요한 합성에서는 중복 비용이 남기 쉽습니다.

입력과 출력

입력은 겹친 삼각형의 화면 범위와 재질·깊이 조건입니다. 결과는 위치별로 실행되는 표면 처리와 합성이며, 비용은 겹침뿐 아니라 각 처리의 계산량에도 달려 있습니다.

작동 과정

삼각형이 화면을 덮으면 표면 후보가 생깁니다. Opaque는 불투명, Masked는 마스크로 남길 부분을 결정하고, Translucent는 뒤의 색과 섞는 표현입니다. 큰 풀 카드의 빈 부분도 어느 영역을 버릴지 마스크를 확인해야 할 수 있습니다. 잎이 작은데 카드가 크면 잎 바깥의 넓은 영역까지 후보를 만듭니다. 여러 카드나 투명 층이 겹치면 비슷한 화면 위치를 반복 처리합니다. 불투명 깊이 제거, 마스크와 반투명 조건에 따라 실제 실행과 생략의 정도가 달라집니다.



외부 진단 화면 · Android Developers가 소개한 Unreal Editor의 Quad Overdraw 시각화입니다.

겹친 영역의 진단색과 아래 범례를 같이 봅니다. 작은 화면 묶음에서의 처리 비효율을 찾는 화면이며 실제 GPU 시간은 아닙니다. 원본은 이전 에디터 UI로 UE 5.8의 메뉴 배치는 다를 수 있습니다.

Android Developers · Google · 출처 [48]

원본 전체 유지 · 투명 배경은 흰색으로 표시 · CC BY 4.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

풀·나뭇잎·연기·머리카락에서 메시의 빈 공간과 겹침을 줄일 때 필요합니다. 윤곽을 다듬어 삼각형을 늘려도 화면 낭비가 줄어 전체가 빨라지는 경우를 설명합니다.

성능과 제작 비용

겹치는 면적·층 수·재질 계산이 함께 비용을 만듭니다. Shader Complexity와 Quad Overdraw는 원인 후보를 보는 진단이며 실제 실행 시간과 같은 단위가 아닙니다.

이 예제의 목표

삼각형을 더 쓴 카드가 빈 화면 영역의 처리를 줄일 수 있는지 확인합니다.

실습 절차

같은 잎 텍스처를 쓰는 큰 사각형 카드와 윤곽을 다듬은 카드를 만듭니다. 잎의 실제 크기·배치·재질·그림자는 같게 두고 가까운 구도에서 비교합니다. Shader Complexity와 가능한 Quad Overdraw를 본 뒤 진단을 끄고 GPU 시간을 기록합니다.

예상 결과와 이유

윤곽을 다듬은 버전은 빈 영역의 진단 부담이 줄 것으로 예상합니다. 픽셀 처리가 병목이라면 삼각형이 늘어도 실행 시간이 줄 수 있습니다.

결과를 어떻게 해석할까

진단색 감소는 낭비 영역을 찾는 단서입니다. 실행 시간과 같은 단위가 아니므로 비용 감소는 별도로 확인합니다. 원거리에서도 같은 결론인지 다시 봅니다.

이 챕터에서 기억할 것

Overdraw는 같은 화면 영역을 반복 처리하는 문제입니다. 폴리곤 수와 함께 화면 면적·빈 공간·겹친 층을 봅니다.

이해한 뒤 확인할 조건

Quad Overdraw는 작은 화면 묶음의 처리 비효율도 보여 줍니다. 일반 Overdraw와 완전히 같은 지표는 아니며 Nanite는 전용 시각화를 함께 사용합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[02] Render Dependency Graph

[07] Viewport Modes

[19] Nanite Virtualized Geometry

07 PBR과 재질 입력

무엇인가

PBR은 Physically Based Rendering으로, 물질과 빛의 상호작용을 물리적으로 그럴듯하게 근사하는 렌더링 접근입니다. 아트 제작에서는 재료의 특성을 입력하고 장면의 빛에 일관되게 반응하도록 만드는 기준이 됩니다.

렌더링에서 맡는 역할

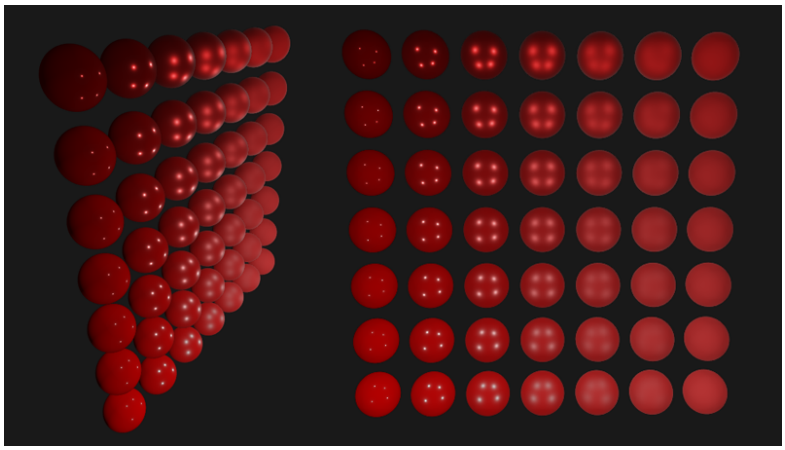
재질의 고유 특성과 특정 장면의 조명을 분리합니다. 한 환경에서만 그럴듯한 색을 그리는 대신, 다른 조명에서도 금속·도장·돌의 구분이 유지되도록 합니다.

입력과 출력

입력은 기본 색, 금속성, 거칠기, 표면 방향 같은 재료 특성과 장면의 빛입니다. 셰이딩 모델이 이를 결합해 관찰 방향으로 나오는 빛을 계산합니다.

작동 과정

재료를 금속·비금속과 표면 상태로 구분합니다. 일반 비금속의 Base Color는 주로 확산 반사색에, 금속의 Base Color는 금속 반사색에 관여합니다. Roughness로 반사가 퍼지는 정도를, 법선으로 미세 표면 방향을 정합니다. Specular 등 입력의 역할은 사용하는 모델에 맞게 적용하며, 밝기 조정용 만능 값으로 쓰지 않습니다. 렌더러가 재료 반응에 광원·환경과 그림자를 결합합니다. 그래서 재료 입력은 같아도 장면의 최종 외관은 달라질 수 있습니다. 조명을 바꿔 재료 구분이 유지되는지 확인합니다.



외부 렌더 이미지 · 금속성과 거칠기를 달리한 구들을 두 시점에서 비교합니다.

오른쪽 배열의 가로 방향에서 거칠기가 변할 때 작은 반사점이 넓어지는 모습을 봅니다. 세로 방향의 금속성 변화도 함께 비교합니다. UE와 광원·톤매핑이 같은 비교 자료는 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [40]
원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

도장 아래 드러난 철, 녹과 오염을 재료별로 나눌 때 필요합니다. Base Color에 특정 방향의 하이라이트를 그려 넣으면 광원이 움직여도 남으므로 물질 흔적과 조명 흔적을 구분합니다.

성능과 제작 비용

PBR이라는 이름이 고정 성능 비용을 정하지는 않습니다. 사용하는 텍스처, 혼합 구조, 셰이딩 모델과 화면 점유율이 실제 계산·메모리 부담을 만듭니다.

이 예제의 목표

금속성·거칠기 입력과 광원 이동의 역할을 나누어 관찰합니다.

실습 절차

같은 구에 Metallic 0과 1, Roughness 0.2와 0.8의 조합을 만듭니다. 처음에는 고정 조명에서 네 구를 비교하고, 이후 재질을 유지한 채 넓은 광원의 방향만 바꿉니다.

예상 결과와 이유

거칠기 변화는 반사의 퍼짐을, 금속성 변화는 확산·스펙큘러 색 반응을 바꿀 것입니다. 광원을 움직이면 하이라이트도 움직여야 합니다.

결과를 어떻게 해석할까

기본 색에 밝은 얼룩이 고정돼 남는다면 구워 넣은 조명 흔적인지 살핍니다. 금속이 검게 보이면 Metallic부터 임의로 낮추기 전에 반사할 환경이 있는지 확인합니다.

이 챕터에서 기억할 것

재료의 고유 반응을 입력하고 조명 계산에 맡기는 것이 PBR 제작의 중심입니다. 조명 하나에만 맞춘 보정을 피합니다.

이해한 뒤 확인할 조건

순수 재료의 Metallic은 0 또는 1을 기준으로 하고 경계의 필터링 등에는 중간값이 생깁니다. sRGB·데이터 채널 패킹은 입력 저장의 심화 항목으로 분리합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[08] Physically Based Materials

[09] Using Fresnel

08 BRDF와 반사 분포

무엇인가

BRDF는 들어오는 빛의 방향에 대해 특정 방향으로 얼마나 반사되는지를 나타내는 함수입니다. 표면에서 바로 일어나는 반사를 설명하며, 표면 내부를 통과하는 모든 빛 전달을 뜻하지는 않습니다.

렌더링에서 맡는 역할

재질 속성의 숫자가 실제 반사 분포로 바뀌는 연결입니다. 거칠기·표면 방향과 빛의 방향이 하이라이트의 폭과 세기를 바꾸는 이유를 설명합니다.

입력과 출력

입력은 빛과 관찰 방향, 법선과 재질 속성입니다. BRDF는 방향별 반사 반응을 주고, 렌더러는 들어오는 빛의 양과 결합해 표면에서 나오는 빛을 계산합니다.

작동 과정

빛이 표면에 어떤 방향으로 도달하는지와 카메라가 어느 방향에서 보는지 정합니다. 같은 재료도 이 두 방향의 관계가 바뀌면 반사가 달라집니다. 미세면 모델은 작은 표면 방향들의 분포로 거친 표면을 근사합니다. 거칠기가 높으면 반사 방향이 넓게 퍼지고, 표면 방향은 반사가 나타나는 위치를 바꿉니다. 여러 방향에서 들어오는 광원의 기여를 모아 관찰 방향의 색을 얻습니다. 넓은 광원도 넓은 하이라이트를 만들 수 있으므로 재질과 광원의 영향을 구분합니다.



외부 개념도 · 미세한 표면 방향의 차이가 반사광의 퍼짐에 영향을 주는 모습입니다.

거친 표면에서는 여러 방향으로, 매끈한 표면에서는 더 모인 방향으로 반사하는 차이를 봅니다. 확대된 개념도이며 실제 메시의 모든 미세 면을 직접 만드는 뜻은 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [41]
원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

젖은 돌·거친 콘크리트·도장 금속의 반사를 조절할 때 필요합니다. 같은 넓이의 하이라이트도 거칠기와 광원 크기라는 다른 원인으로 생길 수 있습니다.

성능과 제작 비용

사용하는 모델의 평가, 광원과 화면 범위가 비용에 관여합니다. 거칠기 숫자 하나를 바꾸는 것과 복잡한 층·텍스처 계산을 추가하는 일은 구분합니다.

이 예제의 목표

거칠기와 광원 크기를 따로 바꿔 비슷한 하이라이트 폭의 다른 원인을 확인합니다.

실습 절차

구 하나와 Rect Light를 둡니다. 먼저 광원 크기·위치·밝기를 고정하고 Roughness만 바꿉니다. 원래 거칠기로 돌린 뒤 광원의 가로·세로 크기만 바꿉니다.

예상 결과와 이유

첫 비교에서는 재질의 반사 분포가, 두 번째에서는 반사에 나타나는 광원의 범위가 달라질 것입니다. 둘 다 하이라이트가 넓어질 수 있지만 원인은 다릅니다.

결과를 어떻게 해석할까

하이라이트 폭뿐 아니라 선명도와 주변 분포를 함께 봅니다. 밝기와 노출까지 동시에 바꾸면 두 원인을 구분하기 어려워집니다.

이 챕터에서 기억할 것

BRDF는 방향에 따른 반사 규칙입니다. 재료와 빛을 따로 조절해야 하이라이트를 의도한 원인으로 만들 수 있습니다.

이해한 뒤 확인할 조건

BRDF의 수식과 미세면 가림 항은 보충 주석에서 읽습니다. 이 장의 목표는 수식 암기보다 광원과 재질의 역할을 구분하는 것입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[08] Physically Based Materials

[10] Overview of Substrate Materials

[31] Real Shading in Unreal Engine 4 · Brian Karis

09 Fresnel과 시선 각도

무엇인가

Fresnel은 두 물질의 경계에서 반사되는 빛의 비율이 각도와 물질 특성에 따라 달라지는 현상입니다. 일반 비금속 경계에서는 스치는 각도에서 반사가 강해지는 반응이 대표적입니다.

렌더링에서 맡는 역할

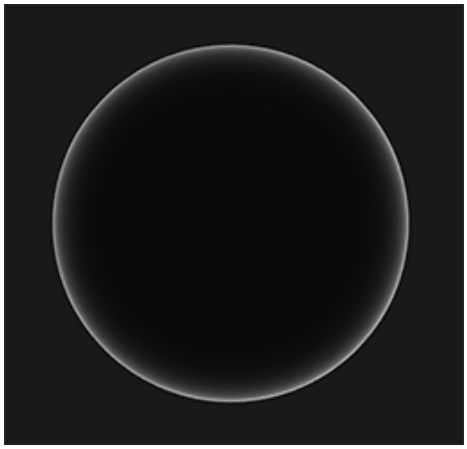
BRDF와 투과 모델 안에서 반사와 투과의 비중을 정하는 데 쓰입니다. 물·유리·코팅이 보는 각도에 따라 다르게 보이는 물리적 근거입니다.

입력과 출력

입력은 경계의 물질 특성과 입사각 등입니다. 결과는 반사율이며 이후 광원·환경·거칠기와 결합해 실제 화면에 보이는 반사가 됩니다.

작동 과정

정면과 비스듬한 입사의 반사율을 구분합니다. F0는 정면 입사에서의 반사율로, 물질 경계의 특성을 표현하는 기준 중 하나입니다. 각도 변화에 따른 반사율을 계산해 표면 반응에 반영합니다. 거친 표면에서는 미세면 방향들이 섞이므로 단순한 화면 가장자리 선 하나로 나타나는 현상은 아닙니다. 렌더러가 반사할 환경과 빛을 결합합니다. 한편 UE의 Fresnel 노드는 시선과 표면 방향으로 마스크를 만들 수 있어, 물리 반사 외의 림 효과에도 사용됩니다.



외부 시각화 이미지 · 구의 중심과 실루엣 부근에서 Fresnel 반사 계수가 달라지는 예입니다.

중심보다 테두리가 밝습니다. 곡면의 테두리에서 시선이 표면에 더 비스듬하게 닿기 때문입니다. 물체의 완성 렌더가 아닌 반사 계수의 시각화입니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [41]
원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

물과 유리의 반사를 자연스럽게 조정하거나, 별도의 가장자리 발광을 설계할 때 필요합니다. 기본 PBR 반사에 포함된 각도 반응과 추가한 아트 효과를 구분해야 중복 보정을 피할 수 있습니다.

성능과 제작 비용

기본 재질의 각도 반응과 추가 텍스처·발광·반투명 기능의 비용은 다릅니다. Fresnel 노드 하나의 외관을 전체 유리 렌더링 비용과 동일시하지 않습니다.

이 예제의 목표

각도 의존 반사와 Fresnel 발광 마스크의 차이를 확인합니다.

실습 절차

반사할 환경이 있는 물 또는 유리 표면을 준비합니다. 카메라 각도만 바꿔 정면과 낮은 시점의 반사를 봅니다. 이후 별도 재질에 Fresnel 노드의 출력을 Emissive에 연결하고 조명을 줄여 비교합니다.

예상 결과와 이유

기본 표면에서는 각도에 따라 환경 반사의 비중이 달라질 것입니다. 발광 마스크는 환경이 어두워져도 스치는 부분을 밝힐 수 있습니다.

결과를 어떻게 해석할까

두 결과가 비슷해 보여도 빛을 만드는 원인이 다릅니다. Fresnel 노드의 마스크를 실제 물질의 반사율 전체와 동일시하지 않습니다.

이 챕터에서 기억할 것

Fresnel은 각도에 따른 반사율 변화입니다. 가장자리 발광 마스크와 물질의 실제 반사를 같은 것으로 취급하지 않습니다.

이해한 뒤 확인할 조건

F0는 정면 입사에서의 반사율입니다. 굴절률과 금속의 파장별 반응 같은 세부는 공식 PBR 자료에서 확장합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[08] Physically Based Materials

[09] Using Fresnel

[10] Overview of Substrate Materials

[31] Real Shading in Unreal Engine 4 · Brian Karis

10 셰이더와 머티리얼

무엇인가

셰이더는 GPU에서 실행하는 계산 프로그램입니다. 머티리얼은 표면이 어떻게 보이고 빛에 반응할지 작성하는 엔진의 제작 체계이며, 머티리얼 그래프는 셰이더 계산으로 변환되는 입력 중 하나입니다.

렌더링에서 맡는 역할

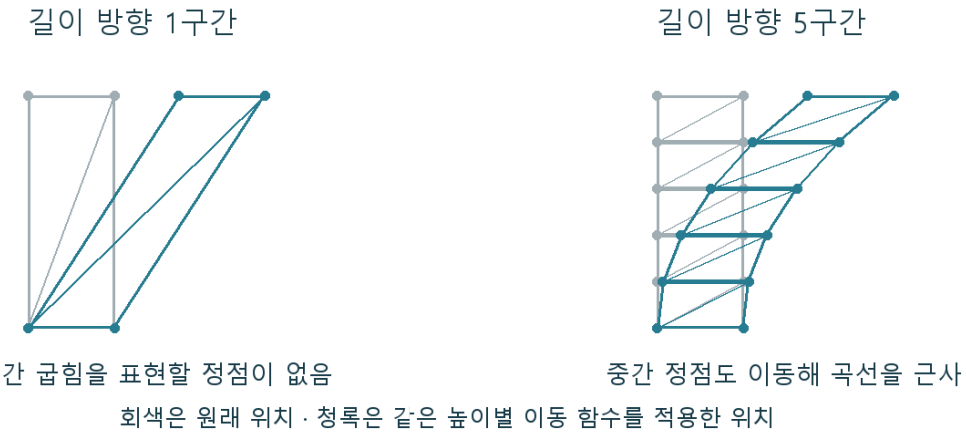
지오메트리·텍스처·재질 속성을 실제 화면 계산으로 연결합니다. 정점의 위치를 바꾸는 일, 표면 값을 구하는 일과 화면 이미지를 처리하는 일은 서로 다른 단계의 프로그램으로 실행될 수 있습니다.

입력과 출력

입력은 정점 속성·텍스처·파라미터·장면 데이터 등입니다. 출력은 단계에 따라 변환된 위치, 표면 속성, 색 또는 다른 계산 데이터가 됩니다.

작동 과정

그래프를 작성하면 엔진이 사용 경로와 설정에 맞는 코드를 구성하고 컴파일합니다. 컴파일은 실행할 프로그램으로 바꾸는 준비 과정입니다. 설정 조합에 따라 서로 다른 셰이더 순열이 필요할 수 있습니다. 정점 셰이더는 위치와 속성 등을 처리합니다. WPO는 기존 정점 위치에 이동량을 더하는 머티리얼 입력입니다. 같은 함수라도 정점 배치가 다르면 표현할 수 있는 굽힘이 달라집니다. 픽셀 셰이더는 표면 후보에서 텍스처와 재질 계산을 수행합니다. 컴퓨트 셰이더는 그림이나 버퍼를 일반적인 병렬 계산으로 처리하는 데 쓰입니다. 그래프의 노드 전체가 픽셀 셰이더 하나와 항상 일대일로 대응하지는 않습니다.



직접 제작 기하 예시. 정점 사이를 직선으로 잇기 때문에 같은 변위 함수라도 정점 배치에 따라 굽힘 표현이 달라집니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

식생 변형은 정점, 표면 혼합은 주로 재질 평가, 후처리는 화면 계산을 바꿉니다. 어떤 입력이 어느 일을 늘리는지 알아야 텍스처 해상도로 형상 문제를 고치려는 혼동을 피할 수 있습니다.

성능과 제작 비용

정점·픽셀 수, 읽는 데이터와 실행 연산이 GPU 비용을 만듭니다. 컴파일 대기와 쿠킹 준비 시간은 실행 비용과 별개의 지표입니다. 노드 수만으로 실제 실행 시간을 단정하지 않습니다.

이 예제의 목표

같은 변위 함수에서 정점의 구간 수가 굽힘에 미치는 영향을 확인합니다.

실습 절차

길이 방향으로 1구간인 카드와 여러 구간인 카드를 만듭니다. 같은 높이 기반 가중치와 같은 바람 함수를 연결합니다. 변형 크기를 고정하고 옆면 실루엣과 바닥 그림자를 비교합니다.

예상 결과와 이유

여러 구간 카드는 중간 정점도 이동해 휘어 보이고, 1구간 카드는 중간 굽힘을 표현하지 못할 것입니다. WPO를 끄면 두 카드가 원래 형상으로 돌아옵니다.

결과를 어떻게 해석할까

외관 비교 뒤 정점 수와 그림자 조건을 기록하고 비용을 따로 봅니다. 움직임이 화면 가장자리에서 사라지면 메시를 감싸는 판정 범위인 Bounds도 확인합니다.

이 챕터에서 기억할 것

셰이더는 GPU의 계산이고 머티리얼은 그중 표면 반응을 작성하는 체계입니다. WPO는 정점 계산을 확인하는 예제이며 셰이더 전체의 정의는 아닙니다.

이해한 뒤 확인할 조건

셰이더 순열은 설정 조합별로 준비하는 프로그램 변형입니다. 실행 중 WPO 비용과 컴파일·쿠킹 준비 비용은 다른 문제이므로 이 실습의 결론에 섞지 않습니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[02] Render Dependency Graph

[11] Material Editor UI

[12] Material Inputs

[62] Graphics Programming Overview · Epic

11 Substrate와 물질 구성

무엇인가

Substrate는 UE에서 물질의 빛 반응을 모듈식으로 작성하고 혼합·적층하는 머티리얼 프레임워크입니다. 하나의 코팅 효과 이름이 아니라 여러 물질 표현을 구성하는 체계입니다.

렌더링에서 맡는 역할

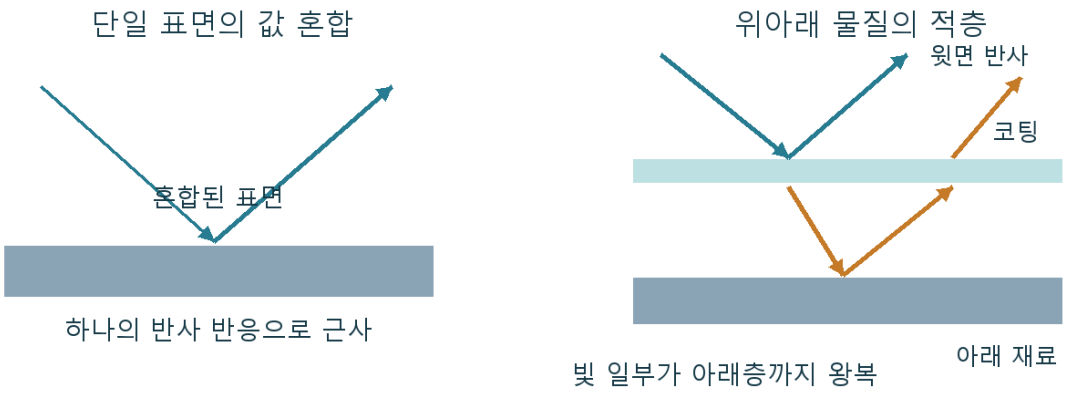
고정된 모델의 제한된 입력만 고르는 대신 물질 단위와 관계를 연결해 복합 표면을 표현합니다. 도장·코팅·물기·산란 같은 재료의 구성을 설명할 수 있게 합니다.

입력과 출력

입력은 Slab 등 물질 단위의 특성과 이를 연결하는 연산자입니다. 구성한 물질 표현을 렌더러가 처리 가능한 형태로 준비하고, 조명에서 표면 반응을 평가합니다.

작동 과정

Slab은 반사·투과·산란 같은 상호작용을 표현하는 물질 단위입니다. BSDF는 경계에서 빛이 어느 방향으로 반사·투과되는지 나타내는 분포 함수의 범주입니다. 혼합은 재료의 비중을 결합하고, 수직 적층은 빛이 위아래 물질을 거치는 관계를 표현합니다. 아래층의 반응은 윗층을 통과하는 빛의 영향을 받으므로 단순한 색·거칠기 평균과 다를 수 있습니다. 구성한 표현은 플랫폼과 예산에 맞는 단순화·저장·평가를 거칩니다. 작성한 층의 의도와 실제 실시간 결과를 여러 조명과 각도에서 확인합니다.



직접 제작한 빛 경로 개념도 · 실제 추적·단순화 구현을 그대로 묘사한 그림은 아닙니다.

직접 제작 개념도. 한 표면의 파라미터 혼합과, 위아래 물질을 통과하는 빛의 관계를 구분합니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

광택 코팅 아래 거친 페인트, 젖은 돌, 재료 경계처럼 서로 다른 반응을 함께 만들 때 필요합니다. 차이가 읽히지 않는 원거리 표면까지 복잡한 구성을 유지할 필요는 없습니다.

성능과 제작 비용

물질의 복잡도·저장량·평가 경로와 단순화가 비용에 관여합니다. Slab 개수만으로 고정된 바이트나 시간을 약속하지 않고 대상 환경의 실제 결과를 확인합니다.

이 예제의 목표

단일 표면 근사와 위아래 적층의 반응을 여러 각도에서 비교합니다.

실습 절차

같은 구에 아래층의 색·거칠기를 고정합니다. 하나는 그 위에 별도 Slab을 수직 적층하고, 다른 하나는 단일 Slab의 값을 조정한 근사 버전으로 둡니다. 카메라와 광원을 유지하며 넓은 각도로 관찰합니다.

예상 결과와 이유

적층 버전에서는 위층 반사와 아래층 반응을 독립적으로 조절할 여지가 생깁니다. 단일 표면은 한 각도에서 비슷해도 다른 각도에서 두 반응을 동시에 맞추기 어려울 수 있습니다.

결과를 어떻게 해석할까

결과가 비슷하다면 조명과 관찰 각도가 층 차이를 드러내는지 확인합니다. 실제 실시간 결과에는 단순화가 개입할 수 있으므로 사용한 설정도 기록합니다.

이 챕터에서 기억할 것

Substrate는 물질 단위와 그 관계를 작성하는 체계입니다. 코팅 예제는 단순 혼합과 적층의 차이를 확인하는 한 사례입니다.

이해한 뒤 확인할 조건

Slab 수만으로 고정 비용을 단정하지 않습니다. 저장 형식·단순화·플랫폼 지원은 버전에 따라 확인할 심화 내용입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[05] Programming with Substrate GBuffer Formats

[06] Decal Materials

[10] Overview of Substrate Materials

12 Translucency와 합성

무엇인가

Translucency는 뒤쪽 장면의 기여를 남기며 표면의 빛 반응과 배경을 결합하는 반투명 표현입니다. 알파 합성, 조명과 굴절이 함께 사용될 수 있지만 각각의 역할은 다릅니다.

렌더링에서 맡는 역할

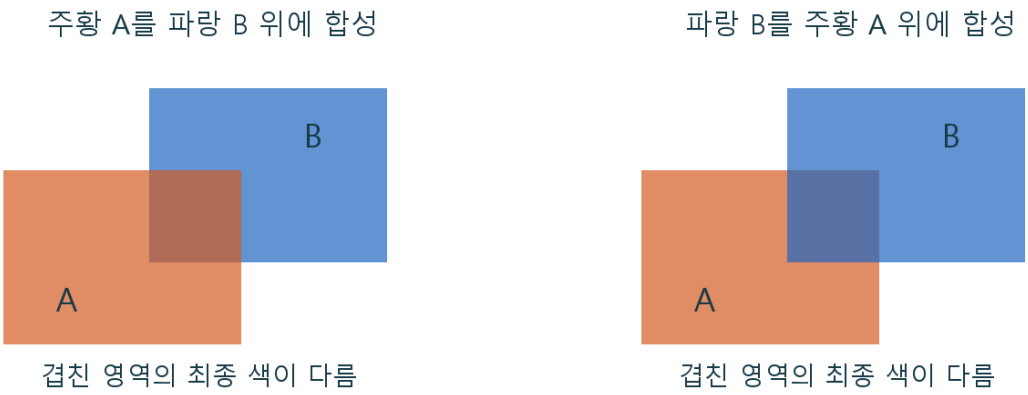
앞면 하나만으로 색이 정해지는 불투명 표면과 달리 여러 표면의 색을 합쳐 화면을 구성합니다. 유리·물·연기가 배경과 어울리도록 만드는 렌더링 영역입니다.

입력과 출력

입력은 반투명 표면의 색·불투명도·조명 반응과 뒤의 장면입니다. 합성한 색이 다음 층 또는 최종 화면의 입력이 됩니다. 굴절은 배경을 읽는 위치나 추적 경로도 바꿀 수 있습니다.

작동 과정

반투명 표면의 재질과 조명을 계산합니다. 값이 작아진 Base Color와 낮은 알파는 같은 조절이 아니며 어떤 합성·셰이딩 모드를 쓰는지 먼저 정합니다. 기본 알파 합성에서는 앞 색과 뒤 색의 기여를 섞습니다. 순서를 바꾸면 결과가 달라질 수 있으므로 깊이 검사만으로 끝내지 않고 표면의 정렬과 합성 순서가 필요합니다. 굴절이 있다면 배경을 다른 위치에서 읽거나 지원 경로로 추적합니다. 합성 정렬의 오류와 화면 밖 배경을 얻지 못하는 굴절 오류는 다른 원인입니다.



직접 계산한 알파 합성 개념도 · 같은 두 색과 알파 0.7에서 합성 순서만 변경

직접 제작 합성 예시. 두 색과 알파는 같지만 합성 순서가 달라 겹친 색이 달라집니다. 실제 엔진 캡처가 아닌 기본 알파 연산의 비교입니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

겹친 유리창·교차하는 판·연기에서 정렬과 배경 누락을 구분할 때 필요합니다. 외관이 이상하다고 재질색부터 바꾸기보다 표면 구조와 합성 경로를 봅니다.

성능과 제작 비용

넓은 화면 면적·여러 층·조명·굴절 계산이 비용을 만듭니다. 뒤쪽 층의 색도 필요하므로 불투명 앞면처럼 모두 생략하기 어렵습니다.

이 예제의 목표

굴절을 끈 두 판으로 합성 순서와 교차 오류를 분리해 관찰합니다.

실습 절차

굴절을 끈 서로 다른 색의 반투명 판 두 개를 둡니다. 판을 떨어뜨린 상태와 교차시킨 상태에서 카메라만 천천히 이동합니다. 불투명 판으로 바꾼 비교도 남깁니다.

예상 결과와 이유

단순히 앞뒤로 떨어진 경우보다 교차한 경우에 겹침 순서가 갑자기 바뀌거나 일부 영역이 어색해질 가능성이 높습니다. 불투명 비교에서는 앞 표면의 깊이가 우선합니다.

결과를 어떻게 해석할까

굴절 없이도 오류가 나면 먼저 정렬·교차 구조를 조사합니다. 굴절을 켠 뒤 생기는 화면 가장자리 누락은 별도 실험으로 다룹니다.

이 챕터에서 기억할 것

반투명은 뒤의 색을 포함해 화면을 만드는 처리입니다. 합성·정렬·굴절을 구분해야 오류와 비용의 원인을 찾을 수 있습니다.

이해한 뒤 확인할 조건

모든 반투명 모드가 같은 정렬과 추적을 사용하지는 않습니다. 이 장의 기본 알파 합성을 이해한 뒤 프로젝트의 선택 경로를 확인합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[13] Lit Translucency

[14] Using Refraction

13 SSS와 표면 아래 산란

무엇인가

SSS는 Subsurface Scattering으로, 빛이 표면 안으로 들어가 내부에서 흩어지고 흡수된 뒤 다시 나오는 현상입니다. 표면에서 즉시 반사되는 빛과 구분되는 전달입니다.

렌더링에서 맡는 역할

피부·왁스·대리석 등에서 부드러운 음영과 내부 빛 전달에 의한 색을 표현합니다. 표면 반사만으로는 만들기 어려운 재료의 밀도감을 담당합니다.

입력과 출력

입력은 선택 모델의 산란·흡수 특성, 스케일·두께 관련 정보와 조명입니다. 모델이 내부 전달을 근사해 표면에서 나오는 빛의 색과 분포에 반영합니다.

작동 과정

빛 일부가 표면 안으로 들어갑니다. 물질은 색에 따라 빛을 다르게 흡수하고, 내부 산란으로 진행 방향을 바꿉니다. 지나온 거리와 재료 특성에 따라 나오는 빛이 달라집니다. 그래서 같은 재료라도 얇은 가장자리와 두꺼운 몸통의 역광 반응이 다를 수 있습니다. 실시간 모델은 이 전달을 여러 방식으로 근사합니다. 모든 광선을 물질 내부에서 실제 추적하는 것은 아니며, 필요한 입력과 근사 조건은 선택 모델에 달려 있습니다.



같은 물질이어도 빛이 지나온 거리와 산란에 따라 나오는 빛이 달라집니다.
직접 제작 현상 개념도 · 특정 UE 산란 모델이 이 경로를 그대로 추적한다는 뜻은 아닙니다.

직접 제작 빛 전달 개념도. 얇고 두꺼운 부분에서 달라질 수 있는 내부 경로를 비교합니다. 실제 결과는 재료와 선택한 산란 모델에 달려 있습니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

왁스의 얇은 부분, 피부의 부드러움과 일부 돌의 투광감을 만들 때 필요합니다. 알파를 내려 배경을 비추는 처리와 내부 빛 전달을 혼동하지 않습니다.

성능과 제작 비용

선택한 산란 모델, 화면 범위와 추가 계산이 비용에 관여합니다. 재료의 스케일과 두께를 틀리게 두고 산란색만 보정하면 다른 조명에서 의도가 무너질 수 있습니다.

이 예제의 목표

한 산란 모델에서 얇은 부분과 두꺼운 부분의 역광 반응을 비교합니다.

실습 절차

얇은 돌기와 두꺼운 몸통이 있는 형상에 같은 산란 재질을 적용합니다. 한 가지 SSS 모델을 고정하고 전면광과 역광을 각각 관찰합니다. 일반 불투명 표면 재질과도 비교합니다.

예상 결과와 이유

역광에서 얇은 부분의 빛 전달이 두꺼운 몸통과 다르게 보일 것으로 예상합니다. 일반 표면 재질은 같은 내부 산란 반응을 만들지 못할 것입니다.

결과를 어떻게 해석할까

예상 차이가 없으면 선택 모델이 두께를 어떻게 근사하거나 입력받는지 확인합니다. 모델의 입력 의미를 먼저 확인하지 않고 Opacity 숫자를 다른 SSS 방식에 옮기지 않습니다.

이 챕터에서 기억할 것

SSS는 빛의 내부 산란입니다. 표면 반사·일반 투명도와 구분하고 재료와 스케일에 맞는 모델을 사용합니다.

이해한 뒤 확인할 조건

Subsurface, Subsurface Profile, Substrate는 같은 입력값이 같은 의미라고 가정할 수 없습니다. 내부 경로 그림도 현상의 설명이며 선택한 모델의 실제 추적을 재현한 것은 아닙니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[10] Overview of Substrate Materials

[15] Subsurface Profile Shading Model

14 Draw Call과 제출 작업

무엇인가

Draw Call은 GPU에 정해진 상태와 지오메트리로 그리기 작업을 실행하도록 제출하는 명령입니다. Actor는 레벨의 오브젝트 단위이고 Draw Call은 실행 작업의 단위이므로 개수가 같지 않습니다.

렌더링에서 맡는 역할

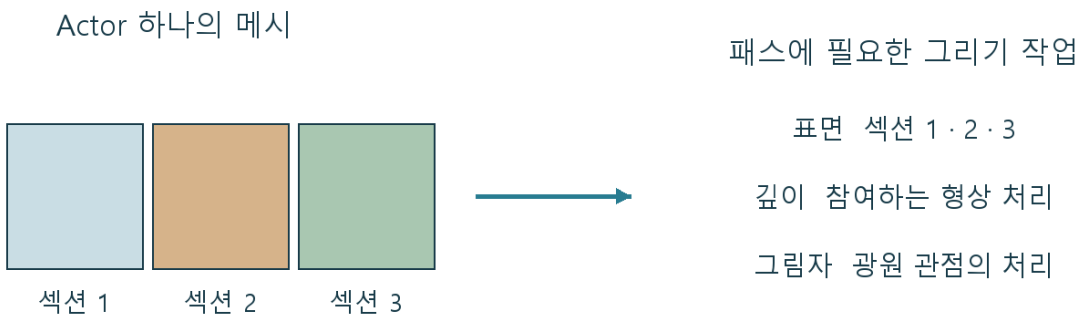
CPU가 준비한 장면을 GPU의 그리기 작업으로 연결합니다. 표면을 어떻게 나누고 어떤 재질과 패스에 참여시키는지가 제출 부담을 만드는 연결 지점입니다.

입력과 출력

입력은 그릴 메시 구역·재질·패스 상태와 관련 데이터입니다. 렌더러는 호환 작업을 준비·묶어 GPU에 제출하며 GPU는 정점과 화면 처리를 실행합니다.

작동 과정

렌더러가 보이는 메시 섹션을 확인합니다. 섹션은 서로 다른 재질 등을 적용할 수 있는 메시 구역이며, 구역별 상태가 다르면 별도 작업이 필요할 수 있습니다. 깊이·기본 표면·그림자처럼 필요한 패스마다 참여 작업을 만듭니다. 같은 메시가 여러 패스에 쓰이므로 물체 하나가 한 번의 명령으로 끝나지 않을 수 있습니다. 호환되는 작업은 배칭이나 인스턴싱으로 묶을 수 있습니다. 실제 제출 수는 자동 묶음과 렌더 경로의 영향을 받으므로 Actor·섹션·실행 명령을 구분해 셉니다.



섹션과 패스가 제출 작업을 나눕니다. 자동 묶음과 경로에 따라 실제 명령 수는 달라집니다.

직접 제작 관계도. Actor 수, 메시 섹션, 패스별 그리기 작업을 나누어 보여 줍니다. 명령 수의 고정 공식은 아닙니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

모듈 건물의 슬롯 수·아틀라스·공유 재질·메시 합체를 결정할 때 필요합니다. 합체로 오브젝트를 줄여도 섹션이 남거나 가시성이 나빠지면 기대한 이득이 없을 수 있습니다.

성능과 제작 비용

명령과 상태를 준비하는 CPU 비용, GPU의 실제 표면 비용은 다릅니다. 호출 수가 줄어도 픽셀 계산이 지배적이면 총 시간이 거의 그대로일 수 있습니다.

이 예제의 목표

Actor 수를 유지하고 섹션 수에 따른 제출 변화를 확인합니다.

실습 절차

자동 묶음의 영향을 통제한 비교 장면에서 외관과 삼각형이 같은 1섹션 메시와 여러 섹션 메시를 준비합니다. 수량·재질 동작·그림자 조건을 맞추고 stat scenerendering 또는 stat rhi의 관련 그리기 수치를 기록합니다.

예상 결과와 이유

섹션 분리가 패스별 제출을 늘리는 조건이라면 여러 섹션 버전의 그리기 수치가 높아질 것입니다. GPU 시간이 반드시 같은 비율로 늘어나는 것은 아닙니다.

결과를 어떻게 해석할까

stat unit의 Draw는 주로 CPU 렌더 스레드 시간이며 호출 개수가 아닙니다. 호출 수가 줄었는지와 실제 병목 시간이 줄었는지는 따로 기록합니다.

이 챕터에서 기억할 것

Draw Call은 그리기 작업의 제출 단위입니다. 물체 수 대신 섹션·패스·묶음과 CPU 제출 시간을 함께 이해합니다.

이해한 뒤 확인할 조건

호환되는 명령을 묶는 배칭과 Nanite의 제출 방식은 결과를 바꿀 수 있습니다. 먼저 사용한 경로를 고정하고 비교합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[03] Introduction to Performance Profiling and Configuration

[16] Mesh Drawing Pipeline

15 Instancing과 반복 배치

무엇인가

Instancing은 공통 메시 같은 데이터를 공유하면서 위치·회전·크기 등의 배치별 정보를 따로 두어 여러 복사본을 처리하는 방식입니다. 같은 형상을 반복하는 데 적합한 데이터·실행 구조입니다.

렌더링에서 맡는 역할

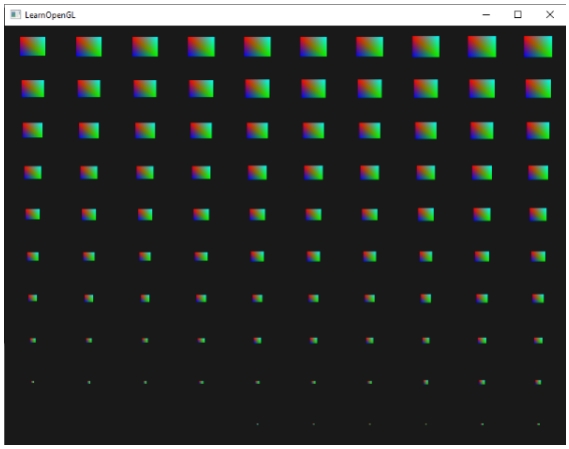
반복 오브젝트를 개별적으로 관리하고 제출하는 부담을 줄일 수 있습니다. 여러 복사본이 보이는 사실 자체를 없애는 것이 아니라 중복 데이터와 준비를 공유하는 역할입니다.

입력과 출력

입력은 공통 메시·재질과 인스턴스별 Transform 등입니다. 이를 이용해 각 위치에 복사본을 표시하며 각 복사본의 가시 형상과 픽셀은 여전히 처리합니다.

작동 과정

공통 형상과 재질을 준비합니다. Transform은 위치·회전·크기의 변환 정보이며 같은 형상을 장면의 다른 위치에 배치하게 합니다. ISM은 반복 배치를 하나의 Instanced Static Mesh Component에서 관리합니다. 인스턴스별 색 같은 변주는 공유 재질에서 별도의 배치별 데이터로 읽을 수 있습니다. 렌더러가 인스턴스의 가시성과 실행 조건을 고려해 처리합니다. HISM은 공간 계층을 사용하는 확장이며 갱신·LOD·컬링 조건에 따라 적합성이 달라집니다.



외부 렌더 이미지 · 공통 사각형을 인스턴싱하면서 배치별 위치와 크기를 달리한 예입니다.

같은 기본 형상을 공유해도 복사본마다 변환 정보가 다릅니다. OpenGL 학습 예시이며 UE ISM의 호출 수나 속도를 측정한 자료는 아닙니다. 외관을 유지하며 제출 비용이 줄었는지는 목표 장면에서 검증합니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [52]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · CC BY 4.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

울타리 기둥·식생·가로등·군집 소품처럼 같은 형상이 반복될 때 필요합니다. 배치마다 독립 재질을 만드는 대신 공통 데이터와 개별 변주를 나눠 설계합니다.

성능과 제작 비용

관리·제출 부담을 줄일 수 있지만 정점·픽셀·그림자와 위치 갱신은 남습니다. 자동 묶음과 Nanite 조건에 따라 제출 이득은 달라질 수 있습니다.

이 예제의 목표

동일 반복 배치를 개별 Actor와 ISM으로 관리할 때의 차이를 확인합니다.

실습 절차

같은 메시·재질의 기둥 200개를 개별 Actor로 둔 버전과 ISM으로 둔 버전을 준비합니다.
배치·충돌·그림자를 맞추고 같은 카메라에서 Actor·컴포넌트 구성과 CPU·GPU 시간을 비교합니다.

예상 결과와 이유

ISM에서는 독립적으로 관리할 Actor·컴포넌트 수가 줄 것입니다. CPU 관리나 제출이 문제였던 장면이라면 그 구간의 개선을 기대할 수 있습니다.

결과를 어떻게 해석할까

GPU 픽셀 비용이 지배하면 전체 프레임 변화는 작을 수 있습니다. 엔진의 자동 인스턴싱 때문에 제출 차이가 작아도 데이터 관리 방식의 차이는 남습니다.

이 챕터에서 기억할 것

인스턴싱은 공통 데이터를 공유하는 반복 처리입니다. 복사본의 화면 계산까지 한 번으로 사라지는 것은 아닙니다.

이해한 뒤 확인할 조건

HISM은 공간 계층을 사용하는 확장입니다. 잦은 위치 갱신과 컬링·LOD·Nanite 조건에 따라 선택이 달라지므로 별도 성능 과제로 다룹니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[16] Mesh Drawing Pipeline

[17] Instanced Static Mesh Component

16 LOD와 화면 크기

무엇인가

LOD는 Level of Detail로, 화면에서 필요한 정도에 맞춰 표현의 세부 수준을 바꾸는 방식입니다. 전통적인 Static Mesh LOD는 미리 만든 대체 메시를 선택해 사용합니다.

렌더링에서 맡는 역할

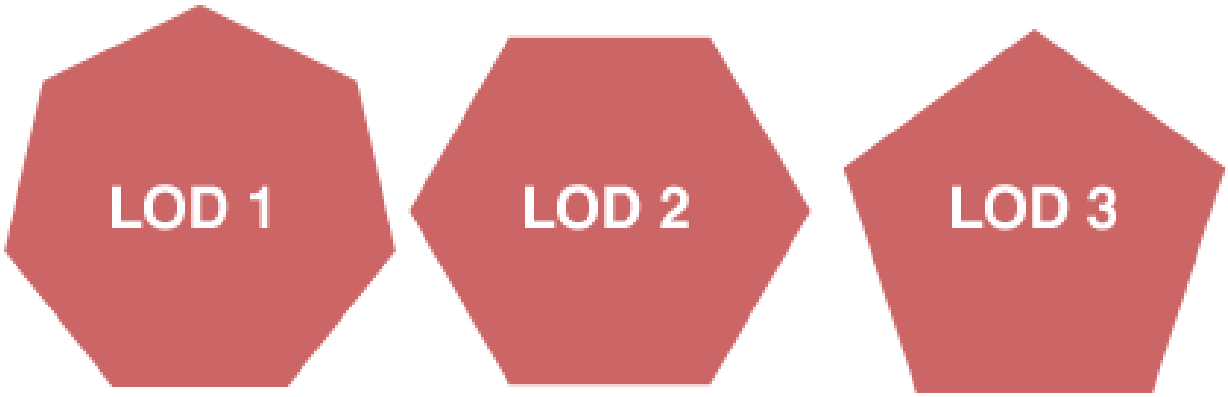
화면에 거의 기여하지 않는 디테일에 쓰는 작업을 줄이면서 필요한 외관을 유지합니다. 단순히 먼 물체를 대충 만드는 것이 아니라 화면 정보에 자원을 배분하는 원리입니다.

입력과 출력

입력은 단계별 메시와 화면 크기·설정입니다. 선택한 단계의 형상과 재질 구성이 렌더링에 사용됩니다. 거리 외에 물체 크기와 카메라 시야각도 선택에 영향을 줍니다.

작동 과정

원본을 기준으로 작은 부품과 면을 줄인 대체 표현을 준비합니다. 실루엣·큰 간격·그림자에 중요한 형태를 먼저 구분합니다. 렌더러가 화면 점유율 등의 기준으로 표현 단계를 선택합니다. 같은 거리라도 큰 물체와 작은 물체 또는 다른 FOV에서는 필요한 디테일이 다릅니다. 카메라 이동으로 단계가 바뀝니다. 두 표현의 윤곽·노멀·재질 차이가 크면 전환 순간 튀므로 정지 외관뿐 아니라 이동과 그림자도 확인합니다.



외부 비교 그림 · 비슷한 실루엣을 더 적은 변으로 근사한 세 가지 LOD입니다.

오른쪽으로 갈수록 위쪽 윤곽이 각지게 바뀝니다. 원본의 LOD 1·2·3 표기는 해당 그림의 번호이며 UE에서 흔히 시작하는 LOD0 번호와 구별합니다.

Sirotk · Wikimedia Commons · 출처 [42]
원본 전체 유지 · 비율 유지 · CC BY-SA 3.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

난간의 나사와 베벨, 먼 건물의 장식처럼 플레이 거리에서 읽히지 않는 부분을 줄일 때 필요합니다. 비율로 일괄 감소하기보다 남길 시각 정보를 먼저 정합니다.

성능과 제작 비용

정점·삼각형·섹션과 재질을 줄일 수 있지만 픽셀 비용이 같으면 총 이득은 작을 수 있습니다. 전환 중 두 표현이 처리되는 조건도 별도로 확인합니다.

이 예제의 목표

카메라 이동에서 LOD가 잃는 디테일과 남겨야 할 윤곽을 판정합니다.

실습 절차

난간의 원본과 나사·작은 베벨을 줄인 LOD를 준비합니다. 같은 구도에서 강제 LOD로 외관을 비교한 뒤, Screen Size 전환을 켜고 카메라를 천천히 이동합니다.

예상 결과와 이유

먼 구도에서는 작은 부품의 차이가 줄어들지만 실루엣을 많이 바꾸면 전환 순간 형태가 튈 것입니다. 가까운 구도에서는 단순화 손실이 더 잘 보일 것입니다.

결과를 어떻게 해석할까

정지 비교에서 비슷해도 이동 중 전환과 그림자 변화를 확인합니다. 줄인 삼각형 수가 아니라 잃은 시각 정보가 허용되는지를 먼저 판단합니다.

이 챕터에서 기억할 것

LOD는 필요한 화면 정보를 남기며 세부를 선택하는 방식입니다. 원본 개수보다 플레이 거리에서의 외관과 비용을 기준으로 설계합니다.

이해한 뒤 확인할 조건

픽셀 재질이 병목이면 삼각형 감소만으로 전체 이득이 작을 수 있습니다. 디더 전환과 재질 단순화는 다음 단계의 선택입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[17] Instanced Static Mesh Component

[18] Optimizing LOD Screen Size Per Platform

17 Nanite와 가상화 지오메트리

무엇인가

Nanite는 높은 디테일의 메시를 작은 묶음과 계층으로 준비하고, 화면에 필요한 세부를 선택·공급하는 UE의 가상화 지오메트리 시스템입니다.

렌더링에서 맡는 역할

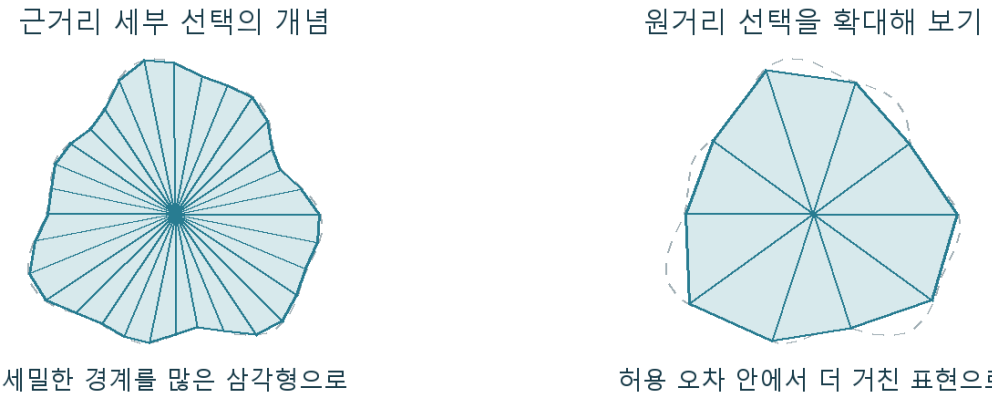
고밀도 원본을 항상 같은 삼각형 집합으로 그리는 대신 가시성과 허용 오차에 맞춰 처리합니다. 전통 수동 LOD의 일부 역할을 다른 구조로 수행하지만 재질·조명 전체를 대신하지는 않습니다.

입력과 출력

입력은 Nanite로 준비한 메시, 배치와 카메라 조건입니다. 선택된 클러스터와 필요한 데이터가 전용 가시성·래스터 처리에 사용됩니다.

작동 과정

빌드 과정에서 메시를 삼각형 묶음인 클러스터와 그 계층으로 구성합니다. 여러 세부 수준을 선택할 수 있는 기반을 만듭니다. 런타임에 가시성과 화면 오차를 기준으로 필요한 묶음을 선택합니다. 작게 보이는 부분은 더 거친 표현으로 충분할 수 있으므로 원본 삼각형 수가 그대로 매 프레임 표시 수가 되지 않습니다. 필요한 메시 데이터를 스트리밍으로 공급하고 전용 경로로 화면에 표시합니다. 선택된 형상의 실루엣과 얇은 면의 보존은 실제 출력 조건에서 확인합니다.



직접 제작 설명도 · 실제 Nanite 클러스터 분할이나 측정 개수를 재현한 그림은 아닙니다.

직접 제작 세부 선택 개념도. 원거리 결과도 비교를 위해 같은 크기로 확대했습니다. 줄인 형상과 원래 경계의 차이를 봅니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

바위·절벽·건축의 고밀도 디테일을 활용하고 에셋 예산을 판단할 때 필요합니다. 얇은 잎·촘촘한 겹침·정점 변형은 별도로 적용 적합성을 확인합니다.

성능과 제작 비용

인스턴스와 보이는 클러스터·겹침·재질·스트리밍이 비용을 만듭니다. 원본 폴리곤 제한이 완화돼도 조명·그림자와 화면 중복 계산은 남습니다.

이 예제의 목표

원본을 바꾸지 않고 화면 크기에 따라 선택되는 Nanite 세부를 관찰합니다.

실습 절차

Nanite 바위 하나를 고정하고 카메라를 가까이에서 멀리 이동합니다. Nanite Visualization의 Triangles와 Clusters를 각각 관찰합니다. 원본 메시를 수정하지 않은 채 선택된 세부가 어떻게 달라지는지 기록합니다.

예상 결과와 이유

거리에 따라 선택되는 삼각형과 클러스터가 달라지면서 최종 실루엣은 가능한 범위에서 유지될 것입니다. 표시색은 재질색이 아니라 선택 단위를 구분하는 진단입니다.

결과를 어떻게 해석할까

원본 개수와 화면에 선택된 정보를 구분합니다. 자동 선택이 외관에 충분한지는 실제 출력 크기의 실루엣·구멍·그림자로 판단합니다.

이 챕터에서 기억할 것

Nanite는 필요한 지오메트리 세부를 선택하고 공급하는 시스템입니다. 원본 수만으로 비용을 판단하거나 모든 렌더링 비용이 사라진다고 보지 않습니다.

이해한 뒤 확인할 조건

Preserve Area의 잎 면적 보존, 폴백 메시와 플랫폼 지원은 적용 대상에 따라 추가 확인합니다. 본문의 개념도는 선택 원리를 설명하며 실제 클러스터 수나 성능을 재현하지 않습니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[17] Instanced Static Mesh Component

[19] Nanite Virtualized Geometry

18 Culling과 가시성

무엇인가

Culling은 화면을 만드는 데 필요하지 않은 렌더링 후보를 제외하는 과정입니다. 거리, 카메라 범위, 다른 물체에 가려졌는지 같은 기준을 사용합니다.

렌더링에서 맡는 역할

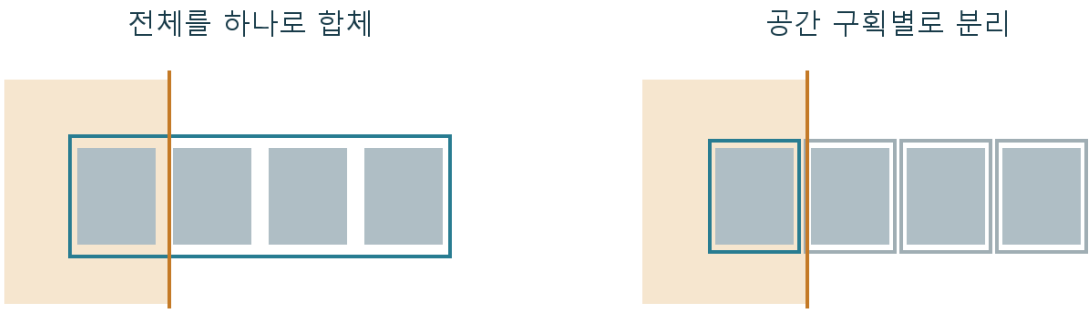
뒤 단계가 처리할 후보를 줄입니다. 모든 물체를 끝까지 계산한 뒤 보이지 않는 색을 버리는 것보다 일찍 제외할 수 있을수록 다른 작업을 줄일 기회가 생깁니다.

입력과 출력

입력은 카메라·물체 범위·가림 정보와 설정입니다. 결과는 이후 렌더링에 넘길 후보 목록이며 메인 카메라와 그림자의 가시 조건은 다를 수 있습니다.

작동 과정

거리 기준과 카메라 시야 공간인 프러스텀 등을 이용해 후보를 줄입니다. 프러스텀 밖 제외와 다른 표면 뒤의 가림 판정은 서로 다른 검사입니다. 물체를 감싸는 상자나 구인 Bounds를 검사 단위로 사용할 수 있습니다. 큰 묶음의 일부가 보이면 전체가 후보에 남을 수 있어 분할 단위가 중요합니다. 깊이 정보 등으로 가려진 후보를 더 줄일 수 있습니다. 실제 검사 순서·지연·세부 단위는 경로에 따라 다르며 Nanite의 클러스터 컬링과 일반 Actor 범위도 구분합니다.



주황 영역은 카메라 범위 안 · 사각 테두리는 Bounds · 같은 건물을 다른 단위로 판정
직접 제작 가시성 개념도 · 실제 원근 시야나 엔진의 모든 컬링 단계를 재현한 그림은 아닙니다.

직접 제작 Bounds 비교. 시야에 일부가 들어온 큰 묶음과, 개별 판정이 가능한 작은 구획의 차이입니다. 가려짐 검사 전의 기본 후보 판정으로 단순화했습니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

건물과 실내 모듈의 합체·분할, 모서리에서의 갑작스러운 등장, WPO 물체의 사라짐을 진단할 때 필요합니다. 제작 편의뿐 아니라 카메라가 가리는 공간 단위도 고려합니다.

성능과 제작 비용

분할하면 제외 기회가 늘지만 관리·검사·제출 작업도 늘 수 있습니다. 메인 화면에 안 보이는 물체의 그림자 기여가 남는지도 함께 봅니다.

이 예제의 목표

합체·구획 버전에서 Bounds와 가림 후보가 어떻게 다른지 확인합니다.

실습 절차

전통 메시로 된 꺾인 복도에서 같은 건물을 통합 버전과 방·구획별 버전으로 준비합니다. Bounds를 표시하고 모서리 앞뒤에서 가시 후보와 관련 렌더링 수치를 비교합니다.

예상 결과와 이유

구획 버전은 완전히 가린 부분을 별도 후보로 제외할 기회가 더 많을 것입니다. 통합 버전은 작은 부분만 보여도 큰 범위가 남을 수 있습니다.

결과를 어떻게 해석할까

메인 화면에 안 보인다고 그림자 작업까지 모두 사라진 것은 아닙니다. 카메라 가시성과 조명 관점의 기여를 구분한 뒤 전체 비용을 비교합니다.

이 챕터에서 기억할 것

컬링은 렌더링 후보를 줄이는 과정입니다. 메시를 묶는 단위와 Bounds가 제외 가능한 범위를 바꿉니다.

이해한 뒤 확인할 조건

Nanite의 클러스터 컬링과 일반 Actor Bounds 판정은 같은 단위가 아닙니다. 이 실습에서는 경로를 고정합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[17] Instanced Static Mesh Component

[19] Nanite Virtualized Geometry

[30] Visibility and Occlusion Culling

19 Mipmap과 텍스처 샘플링

무엇인가

텍스처 샘플링은 UV 등으로 이미지에서 필요한 값을 읽는 작업입니다. Mipmap은 작게 보이는 표면을 안정적으로 읽도록 원본을 단계적으로 축소해 준비한 이미지 계층입니다.

렌더링에서 맡는 역할

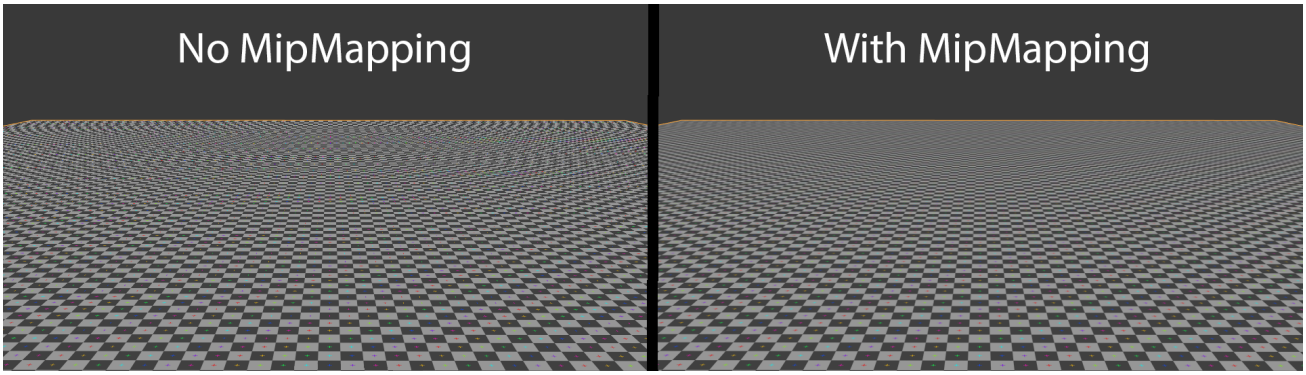
화면 픽셀보다 세밀한 패턴을 적절한 크기로 대표하게 합니다. 원거리 깜빡임·모아레를 줄이고 불필요하게 큰 텍스처 데이터를 읽는 일을 줄이는 데 사용됩니다.

입력과 출력

입력은 텍스처·UV와 화면에서 UV가 변하는 정도, 필터 설정입니다. 읽을 밉과 값들을 선택·혼합해 해당 표면 위치의 재질 입력값을 만듭니다.

작동 과정

원본을 축소해 여러 크기의 밉을 준비합니다. 텍셀은 텍스처 데이터의 한 칸이며, 먼 표면에서는 화면 픽셀 하나가 많은 텍셀을 대표해야 합니다. 화면에서의 UV 변화 등을 이용해 적절한 밉을 선택합니다. 너무 세밀한 원본에서 한 점만 읽으면 작은 이동에도 서로 다른 무늬를 고르기 쉽습니다. 필터링으로 샘플과 밉 사이의 값을 섞습니다. 비스듬한 바닥처럼 한 방향으로 길게 늘어난 범위에는 이방성 필터링이 도움을 줄 수 있습니다.



외부 실제 렌더 비교 · 왼쪽 No MipMapping, 오른쪽 With MipMapping입니다.

수평선에 가까운 먼 바닥에서 왼쪽의 물결무늬와 오른쪽의 안정된 패턴을 비교합니다. 원본은 Blender 렌더이며 필터링도 관여할 수 있으므로 UE에서 밉만 바꾼 통제 실험으로 해석하지 않습니다.

BillyBob CornCob · Wikimedia Commons · 출처 [43]

원본 전체 유지 · 비율 유지 · CC0 1.0

이용 조건

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

먼 바닥 줄눈과 반복무늬, 미세 노멀의 안정성을 조절할 때 필요합니다. 원본 해상도를 늘리는 것과 플레이 화면에서 읽히는 패턴을 만드는 일은 다릅니다.

성능과 제작 비용

밉 체인은 저장 데이터를 더 필요로 하지만 작은 밉 사용은 캐시·대역폭에 유리할 수 있습니다. 실제 상주 메모리는 스트리밍과 압축 등에도 달려 있습니다.

이 예제의 목표

같은 이동에서 밉의 선명도 손실과 시간적 안정성을 함께 관찰합니다.

실습 절차

잔무늬 바닥의 일반 밉 버전과 NoMipmaps 버전을 준비합니다. 원하는 밉이 상주한 조건에서 같은 낮은 시선으로 천천히 이동하며 원거리 영역을 녹화합니다.

예상 결과와 이유

밉이 없는 버전은 먼 패턴의 모아레와 깜빡임이 커지고, 일반 밉은 일부 선명도를 줄이면서 더 안정될 것으로 예상합니다.

결과를 어떻게 해석할까

정지 화면의 선명도만 고르지 않습니다. 비스듬한 바닥에서는 이방성 필터링도 관여하므로 그 설정을 고정한 뒤 밉 차이를 봅니다.

이 챕터에서 기억할 것

밉은 화면 크기에 맞는 대표 정보를 준비하고 샘플링은 그것을 읽는 과정입니다. 선명도와 이동 안정성을 함께 판단합니다.

이해한 뒤 확인할 조건

밉 체인의 데이터 증가와 캐시·대역폭 이득은 성능의 확장 주제입니다. 밉 선택과 실제 메모리 상주는 CH20에서 구분합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[20] Texture Asset Editor

[21] Texture Streaming Overview

20 Texture Streaming

무엇인가

Texture Streaming은 화면에 필요한 텍스처 밉을 추정하고 메모리 예산 안에서 불러오거나 내보내는 관리 시스템입니다. 원본 텍스처를 항상 최고 해상도로 메모리에 두는 방식과 다릅니다.

렌더링에서 맡는 역할

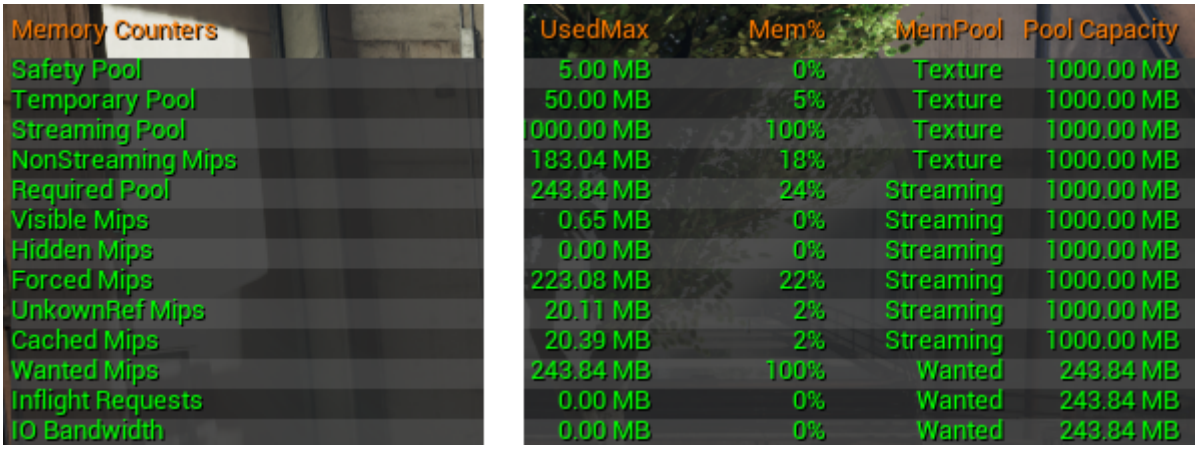
많은 에셋을 사용하는 장면에서 메모리와 로딩을 관리합니다. CH19의 밉 선택이 읽을 세부 수준의 문제라면, 스트리밍은 필요한 데이터가 실제로 준비돼 있는지의 문제입니다.

입력과 출력

입력은 텍스처와 장면 배치·화면 요구량·메모리 예산입니다. 출력은 로드·해제 요청과 실제로 메모리에 상주하는 밉의 상태입니다.

작동 과정

카메라와 물체 크기·거리 등을 이용해 필요한 세부 수준을 추정합니다. 화면에서 원하는 밉과 현재 준비된 밉을 구분합니다. 예산과 우선순위에 맞춰 로드·해제를 결정합니다. 스트리밍 풀은 이 관리에 쓰는 메모리 예산의 한 묶음이며 전체 GPU 메모리와 같은 뜻은 아닙니다. 디스크에서 데이터를 옮겨 요청한 밉을 준비합니다. 이 전송에 시간이 걸리므로 이동 직후 잠시 낮은 밉으로 보이다 선명해질 수 있습니다.



Memory Counters	UsedMax	Mem%	MemPool	Pool Capacity
Safety Pool	5.00 MB	0%	Texture	1000.00 MB
Temporary Pool	50.00 MB	5%	Texture	1000.00 MB
Streaming Pool	000.00 MB	100%	Texture	1000.00 MB
NonStreaming Mips	183.04 MB	18%	Texture	1000.00 MB
Required Pool	243.84 MB	24%	Streaming	1000.00 MB
Visible Mips	0.65 MB	0%	Streaming	1000.00 MB
Hidden Mips	0.00 MB	0%	Streaming	1000.00 MB
Forced Mips	223.08 MB	22%	Streaming	1000.00 MB
UnkownRef Mips	20.11 MB	2%	Streaming	1000.00 MB
Cached Mips	20.39 MB	2%	Streaming	1000.00 MB
Wanted Mips	243.84 MB	100%	Wanted	243.84 MB
Inflight Requests	0.00 MB	0%	Wanted	243.84 MB
IO Bandwidth	0.00 MB	0%	Wanted	243.84 MB

Epic 공식 통계 화면 · stat streaming의 Memory Counters를 발췌하고 항목과 수치 사이의 여백을 줄였습니다.

Memory Counters의 Pool·Required·Visible 등 서로 다른 항목을 구분합니다. 화면의 숫자는 원문 예시이며 권장 예산이 아닙니다. UE 5.5 자료이므로 항목 배치는 현재 빌드에서 확인합니다.

© Epic Games · 공식 문서 · 출처 [54]
원본 통계 표 발췌 · 가운데 여백만 축소 · 값과 항목 유지 · 원저작권과 원문 출처 표시는 유지합니다

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

큰 환경의 텍스처 해상도 배분, 흐린 소품과 팝인, 첫 진입 끊김을 조사할 때 필요합니다. 원본 크기·수요 추정·예산·전송 지연 중 원인을 구분합니다.

성능과 제작 비용

높은 밉의 메모리와 데이터 전송이 부담입니다. 풀을 키우면 다른 리소스 여유가 줄 수 있어 경고를 없애는 것만을 목표로 값을 늘리지 않습니다.

이 예제의 목표

첫 진입과 재방문에서 실제 밉 준비 상태와 선명해지는 지연을 대조합니다.

실습 절차

고정된 이동 경로를 처음 지나갈 때와 곧바로 재방문할 때 비교합니다. stat streaming에서 필요한 밉, 메모리 예산, 상주·요청 상태를 기록하고 같은 위치의 팝인을 녹화합니다.

예상 결과와 이유

처음 데이터가 준비되지 않은 조건에서는 선명해지는 지연이 더 클 수 있습니다. 재방문 때 필요한 밉이 남아 있다면 더 빠르게 선명해질 것입니다.

결과를 어떻게 해석할까

두 번 모두 계속 흐리면 예산·수요 추정·최대 해상도를 봅니다. 재방문 개선만으로 원인을 단정하지 말고 실제 밉의 준비 상태와 대조합니다.

이 챕터에서 기억할 것

텍스처 스트리밍은 필요한 밉을 메모리에 준비하는 관리입니다. 필요한 양·예산·실제 상주량을 구분해 진단합니다.

이해한 뒤 확인할 조건

일반 Texture Streaming은 Virtual Texturing이나 월드 Actor 스트리밍과 같은 기능이 아닙니다. 표의 외부 숫자는 권장 예산이 아닌 항목 설명용입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[20] Texture Asset Editor

[21] Texture Streaming Overview

[22] Texture Streaming Metrics

21 Lumen과 간접광

무엇인가

Lumen은 UE에서 동적 전역 조명과 반사를 계산하는 시스템입니다. 전역 조명은 표면 사이에서 전달되는 빛을 포함하는 조명이며, Lumen은 변화하는 장면의 간접광과 반사를 실시간으로 근사합니다.

렌더링에서 맡는 역할

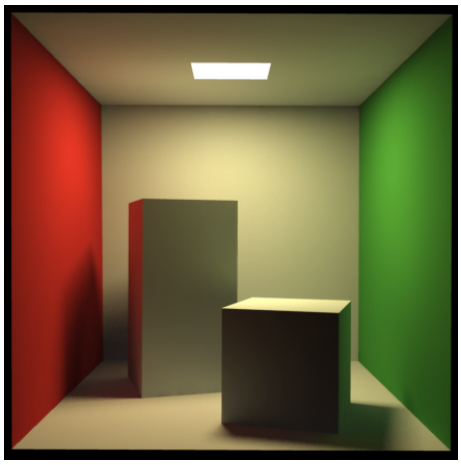
직접광만으로 설명되지 않는 실내 밝기와 색 번짐, 주변 장면의 반사를 만듭니다. 재질은 같은데 주변 표면이나 광원을 바꾸면 외관이 달라지는 빛 전달을 담당합니다.

입력과 출력

입력은 장면의 형상·재질·조명과 카메라 정보입니다. 추적과 조명 조회로 간접광·반사를 구해 표면을 조명하는 데 사용합니다.

작동 과정

광원에서 바로 도달한 빛과 다른 표면을 거쳐 온 빛을 구분합니다. 붉은 벽에서 반사된 빛이 흰 벽을 붉게 비추는 것이 간접광의 예입니다. 화면에 있는 정보로 추적을 수행할 수 있고, 화면만으로 부족한 부분은 별도 장면 표현과 지원되는 소프트웨어·하드웨어 추적으로 보완합니다. 화면 밖 물체도 빛 전달에 참여하기 때문입니다. Surface Cache 같은 표면 표현에서 재질·조명 정보를 조회하고 결과를 갱신·통합합니다. 반사의 조명 평가 방식과 추적 설정은 사용 경로와 품질·비용 조건에 따라 선택합니다.



외부 렌더 참고. 왼쪽 빨강·오른쪽 초록 벽이 주변 중성 표면의 색에 영향을 줍니다. 경로 추적 Cornell box로 간접광 현상을 보여 주며, Lumen 실행 결과는 아닙니다.

Trevorgoodchild · Public domain · 출처 [36]

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

실내 누광·빛 번짐·반사 누락과 재질색을 구분하고 메시 구성·조명 예산을 정할 때 필요합니다. 흰 벽의 붉은 기운을 바로 텍스처에서 상쇄하기 전에 주변 빛 전달을 봅니다.

성능과 제작 비용

장면 표현의 갱신, 추적, 조회와 화면 결과의 통합에 비용이 듭니다. 복잡한 장면·변화·품질 설정에 따라 달라지므로 간접광과 반사 관련 작업을 따로 관찰합니다.

이 예제의 목표

색 벽만 바꿔 흰 벽의 재질색과 간접광을 구분합니다.

실습 절차

한쪽 벽만 색이 있는 작은 방과 흰 수신 벽을 준비합니다. Lumen GI와 노출·직접광을 고정합니다. 색 벽의 Base Color만 빨강에서 중성 회색으로 바꾸고 수신 벽의 Base Color와 Lit를 비교합니다.

예상 결과와 이유

수신 벽의 Base Color는 유지되고 Lit의 색 번짐은 달라질 것입니다. 직접광은 같아도 주변 재료가 간접 조명을 바꾼다는 예상입니다.

결과를 어떻게 해석할까

변화가 없으면 GI 설정과 장면 표현을 먼저 확인합니다. Surface Cache는 추적한 표면의 재질·조명을 조회하는 저장 체계입니다. 누락을 조사할 때 진단하지만 이 장의 첫 결론은 색 번짐의 원인입니다.

이 챕터에서 기억할 것

Lumen은 변화하는 장면의 간접광과 반사를 근사합니다. 재질 입력과 주변 장면에서 전달되는 빛을 나눠 이해해야 올바르게 조정할 수 있습니다.

이해한 뒤 확인할 조건

반사에서 Surface Cache와 Hit Lighting을 비교하는 것은 별도 문제입니다. 반사 모드나 추적 옵션을 이 간접광 실험과 동시에 바꾸지 않습니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[23] Lumen Technical Details

[24] Lumen Performance Guide

22 Virtual Shadow Maps

무엇인가

Virtual Shadow Maps는 그림자 데이터를 필요한 작은 페이지 단위로 관리하는 고해상도 그림자 시스템입니다. 광원 관점에서 어떤 표면이 가려지는지 계산하는 그림자 맵의 가상화 방식입니다.

렌더링에서 맡는 역할

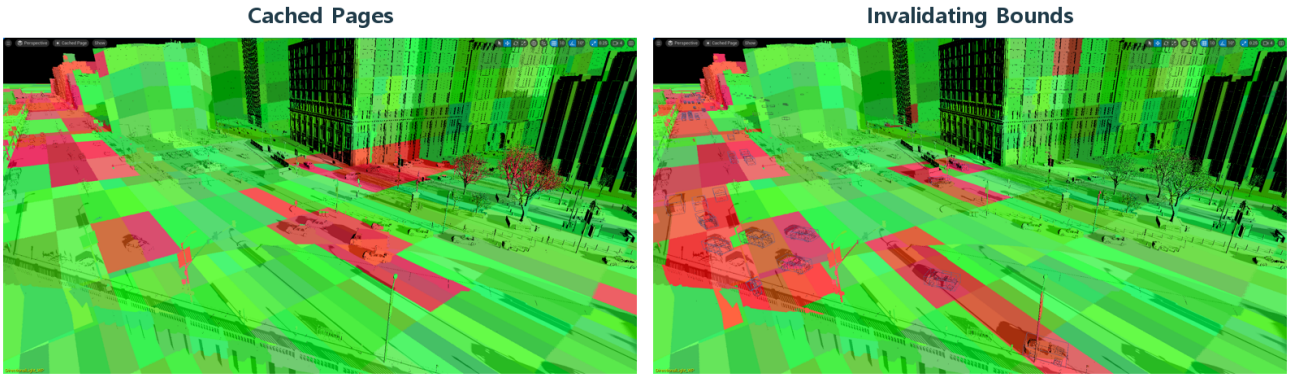
화면에 필요한 그림자 디테일에 데이터를 배분하고, 변하지 않은 결과를 재사용합니다. 해상도와 메모리·갱신 작업을 관리하는 구조이지 그림자를 한 번 그려 영구히 쓰는 기능은 아닙니다.

입력과 출력

입력은 광원·가시 영역·그림자를 만드는 형상과 변화 정보입니다. 필요한 페이지의 깊이를 준비하고 이를 이용해 화면 표면의 그림자 여부를 평가합니다.

작동 과정

현재 시야와 광원에서 필요한 그림자 페이지를 정합니다. 필요한 작은 영역을 할당·준비하므로 거대한 모든 영역을 항상 같은 해상도로 새로 그리지 않습니다. 기존 페이지가 유효하면 재사용합니다. 카메라 이동으로 새 영역이 필요하거나 광원·형상이 변해 결과가 낡으면 새로 그리거나 무효화 후 갱신합니다. 준비한 깊이로 조명 계산 시 가림을 평가합니다. 깊이를 만드는 일과 화면에 그림자를 적용하는 일은 서로 다른 작업으로 관찰할 수 있습니다.



Epic 공식 진단 비교 · 왼쪽은 캐시 페이지, 오른쪽은 무효화를 유발하는 Bounds도 표시한 화면입니다.

이 시각화에서 초록은 재사용, 빨강은 새로 필요하거나 무효화된 페이지입니다. 오른쪽 경계가 어떤 영역에 영향을 주는지 봅니다. UE 5.4 문서의 표시 예이며 비용 측정은 진단 모드를 끈 뒤 수행합니다.

© Epic Games · 공식 문서 · 출처 [56]
동일 범위의 원본을 나란히 배치 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

식생의 바람·이동 소품·광원 변화가 그림자 비용에 미치는 영향을 이해할 때 필요합니다. WPO와 큰 Bounds가 어느 범위의 갱신을 유발하는지 조사합니다.

성능과 제작 비용

필요 페이지의 메모리, 새 페이지·무효화 영역의 렌더링, 그림자 평가가 비용입니다. 정지 장면의 캐시 효과와 움직이는 장면의 갱신 비용을 구분합니다.

이 예제의 목표

정지 카메라에서 WPO만 바꿔 그림자 페이지 갱신의 원인을 확인합니다.

실습 절차

카메라와 광원을 고정한 식생 장면에서 바람 WPO만 켜고 끕니다. VSM의 Cached Page와 무효화 Bounds 시각화를 비교합니다. 이후 진단을 끄고 그림자 생성 시간과 전체 GPU 시간을 기록합니다.

예상 결과와 이유

바람이 켜진 상태는 해당 식생 주변에 지속적인 페이지 갱신이 생길 수 있습니다. 바람을 끄면 정적 영역의 재사용 비중이 높아질 것으로 예상합니다.

결과를 어떻게 해석할까

새로 보인 영역 때문에 만든 페이지와 변형 때문에 다시 만든 페이지를 구분합니다. 시각화 변화가 실제 실행 시간에 얼마나 영향을 줬는지는 별도로 측정합니다.

이 챕터에서 기억할 것

VSM은 필요한 그림자 페이지를 준비하고 유효한 결과를 재사용합니다. 움직임과 판정 범위가 캐시 이득을 바꿉니다.

이해한 뒤 확인할 조건

페이지 색과 명령은 버전별 표시를 확인합니다. 메모리 예산·투영 비용은 갱신 원리를 이해한 뒤 확장할 항목입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[19] Nanite Virtualized Geometry

[25] Virtual Shadow Maps

23 TAA와 프레임 히스토리

무엇인가

TAA는 Temporal Anti-Aliasing으로, 현재와 이전 프레임의 정보를 이용해 경계의 계단 현상과 시간적 깜빡임을 줄이는 안티앨리어싱 방식입니다.

렌더링에서 맡는 역할

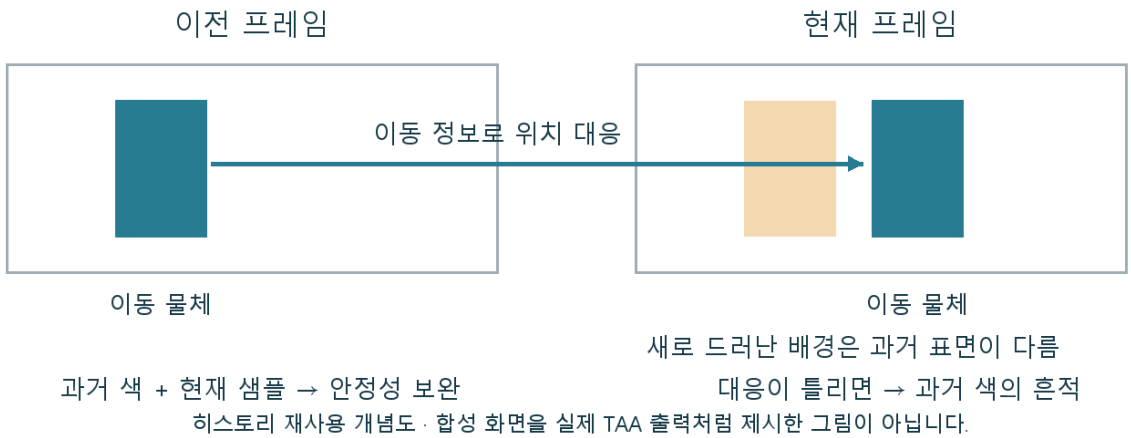
제한된 픽셀 샘플이 놓친 세부를 여러 프레임에 걸쳐 보완합니다. 단일 정지 이미지의 경계뿐 아니라 카메라 이동 중 얇은 패턴의 안정성을 담당합니다.

입력과 출력

입력은 현재 색, 이전 결과인 히스토리, 깊이와 이동 정보 등입니다. 과거 정보를 현재 화면 위치에 대응시키고 신뢰도를 판단해 누적한 색을 만듭니다.

작동 과정

프레임마다 조금 다른 샘플 위치로 장면을 계산합니다. 같은 얇은 선도 여러 위치에서 검사하면서 한 프레임이 얻지 못한 정보를 모을 수 있습니다. 물체·카메라의 이동 정보로 이전 위치와 현재 위치를 대응시킵니다. Velocity는 화면상의 이동 정보이며, 변형 재질에도 올바른 대응이 필요합니다. 맞는 히스토리는 현재 정보와 결합하고 불확실한 과거 정보는 줄이거나 버립니다. 새로 드러난 표면과 잘못된 대응에서는 잔상·불안정성이 나타날 수 있습니다.



직접 제작 시간 흐름도. 물체의 이전 위치를 현재 위치에 대응시키는 과정과 새로 드러난 배경의 문제를 보여 줍니다.

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

철망·가는 가지·WPO 식생과 이동하는 재질의 깜빡임·잔상을 진단할 때 필요합니다. 패턴 크기와 이동 정보를 관리하고 정지·이동·가림 해제를 구분해 관찰합니다.

성능과 제작 비용

현재·과거 버퍼를 읽고 대응·필터링하는 화면 계산과 저장 비용이 있습니다. 화질은 히스토리 조건에도 달려 있어 정지 캡처 한 장으로 품질을 확정하지 않습니다.

이 예제의 목표

철망의 정지·이동·가림 해제를 따로 기록해 누적의 이점과 흔적을 봅니다.

실습 절차

내부 해상도 100%에서 TAA와 AA 없음의 철망을 같은 속도로 이동 촬영합니다. 정지한 뒤, 등속 이동, 전경 물체가 지나가 배경이 드러나는 순간을 각각 기록합니다.

예상 결과와 이유

TAA는 정지·완만한 이동에서 얇은 경계를 더 안정시킬 수 있습니다. 새로 드러나는 영역이나 빠른 움직임에서는 재구성 흔적이 더 잘 보일 수 있습니다.

결과를 어떻게 해석할까

정지 이미지 한 장으로 시간적 품질을 판정하지 않습니다. 변형 식생에서만 흔적이 심하면 WPO와 Velocity 출력도 조사합니다. Velocity는 화면상의 이동을 나타내는 정보입니다.

이 챕터에서 기억할 것

TAA는 과거 프레임을 현재와 대응해 경계를 안정시키는 방식입니다. 누적의 이점과 잘못된 과거 정보의 흔적을 함께 봅니다.

이해한 뒤 확인할 조건

히스토리 거부와 필터 세부는 구현마다 다릅니다. 이 장은 TAA의 원리이며 TSR의 입력 해상도 문제는 다음 장에서 다룹니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

24 TSR과 시간적 재구성

무엇인가

TSR은 Temporal Super Resolution으로, 현재 프레임과 히스토리를 활용해 목표 출력 해상도의 이미지를 재구성하는 UE의 시간적 업스케일러입니다.

렌더링에서 맡는 역할

낮은 내부 해상도로 장면을 계산하면서 높은 출력 해상도를 제공할 수 있게 합니다. 경계를 안정시키는 시간적 누적을 해상도 재구성과 연결하는 시스템입니다.

입력과 출력

입력은 내부 해상도의 장면 색, 이동·깊이 정보와 히스토리 등입니다. 출력은 목표 해상도의 이미지입니다. 입력 픽셀 수와 표시할 픽셀 수는 같지 않을 수 있습니다.

작동 과정

Screen Percentage 등으로 내부 렌더링 해상도를 정합니다. 가로·세로 비율을 함께 낮추면 장면을 계산할 픽셀 수가 더 크게 줄어듭니다. 이동 정보를 이용해 과거 세부를 현재 위치에 대응시키고 현재 샘플과 결합합니다. 단순히 한 장을 늘리는 것보다 시간적으로 얻은 정보를 활용합니다. 신뢰할 수 없는 히스토리를 처리하며 목표 출력으로 재구성합니다. 새로 드러나는 글자·얇은 선·빠른 반사는 입력 정보의 부족을 드러내기 쉽습니다.

Native 4K



1080p → 4K TSR



Epic 공식 TSR 비교 · 왼쪽 네이티브 4K, 오른쪽 1080p에서 4K로 재구성한 결과의 같은 구역을 발췌했습니다.

출력 크기가 같아도 들어온 정보량은 다릅니다. 글자·창틀 같은 세부를 비교하되 정지된 한 장이 이동 품질을 보장하지는 않습니다. UE 5.5 원문 장면의 성능 수치를 이 프로젝트의 예상 이득으로 사용하지 않습니다.

© Epic Games · 공식 문서 · 출처 [58]
원본 일부 발췌·확대 · 색상 유지 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

목표 기기의 화면 품질과 GPU 예산을 맞출 때 필요합니다. 출력 해상도만 보고 같은 화질이라고 결정하지 말고 내부 해상도와 이동 결과를 함께 봅니다.

성능과 제작 비용

장면의 화면 단위 작업을 줄일 수 있지만 TSR 자체의 처리·저장 비용과 고정 비용은 남습니다. 입력 픽셀 감소율이 총 프레임 시간 감소율은 아닙니다.

이 예제의 목표

출력 크기를 유지하고 입력 해상도에 따른 총 시간과 이동 품질을 비교합니다.

실습 절차

출력 크기와 Dynamic Resolution을 고정합니다. TSR을 유지한 채 Screen Percentage 100과 67을 비교합니다. 같은 경로의 글자·철망·반사를 녹화하고 전체 GPU 시간과 TSR 시간을 따로 기록합니다.

예상 결과와 이유

67%는 가로·세로 각각의 비율이므로 입력 픽셀 수는 약 45%가 됩니다. 화면 단위 작업은 줄 수 있지만 재구성 비용과 고정 비용 때문에 전체 시간이 같은 비율로 줄지는 않을 것입니다.

결과를 어떻게 해석할까

정지 상태의 선명도와 이동·가림 해제의 품질을 나눠 판정합니다. TSR 패스가 늘거나 유지돼도 다른 패스의 절감으로 전체가 빨라질 수 있습니다.

이 챕터에서 기억할 것

TSR은 시간적 정보를 이용한 출력 이미지 재구성입니다. 내부 해상도의 절감과 실제 이동 화질·전체 시간을 함께 판단합니다.

이해한 뒤 확인할 조건

외부 비교의 장비·장면 성능을 자신의 예상 이득으로 사용하지 않습니다. TAAU와 다른 업스케일러는 별도 비교 대상입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

25 CPU와 GPU 병목

무엇인가

병목은 프레임의 처리율이나 완료 시간을 제한하는 작업 또는 대기 관계입니다. CPU와 GPU 병목 분석은 어느 쪽의 준비·실행이 실제로 프레임 진행을 제한하는지 찾는 과정입니다.

렌더링에서 맡는 역할

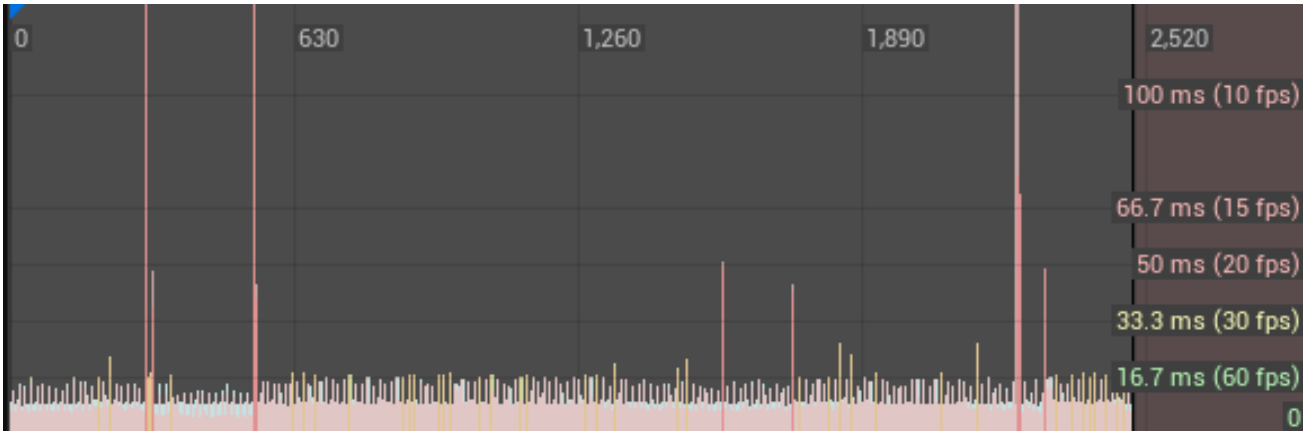
효과가 있는 최적화 대상의 우선순위를 정합니다. 눈에 복잡한 에셋을 줄이는 것과 프레임을 제한하는 일을 줄이는 것은 같지 않으므로 먼저 제한 원인을 확인합니다.

입력과 출력

입력은 반복 가능한 실행 구간과 CPU·GPU·프레임 시간 및 타임라인입니다. 결과는 계산·제출·동기화·로딩 중 어느 작업이 제한 후보인지에 대한 판단입니다.

작동 과정

CPU의 게임 상태 갱신과 명령 준비, GPU의 실행은 여러 프레임에서 겹칠 수 있습니다. 단순히 모든 시간을 더해서 프레임 시간을 구하지 않습니다. stat unit 등으로 Frame·Game·Draw·GPU를 살펴 큰 방향을 정합니다. 프레임 제한과 VSync가 만든 대기라면 계산을 줄여도 FPS가 유지될 수 있습니다. 해상도 같은 영향 범위가 알려진 변수를 바꾸고 반응을 봅니다. 이어 타임라인에서 계산과 기다림, 작업 의존 관계를 확인해 원인을 좁힙니다.



외부 실측 화면 · John Farrow가 기록한 Unreal Insights의 프레임 시간 그래프입니다.

긴 막대가 나타난 구간을 고른 뒤 CPU·GPU 시간축을 열어 원인을 찾습니다. 그래프 자체로 CPU 병목이라고 판정하지 않습니다. 저자의 2024년 프로젝트 예시이며 이 저장소의 측정 결과가 아닙니다.

© John Farrow · Unrealcode.net · 출처 [59]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 원저작권과 원문 출처 표시는 유지합니다

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

폴리곤과 재질을 줄여도 FPS가 그대로일 때, 첫 진입만 끊길 때, CPU 제출이 무거운 대규모 배치에서 필요합니다. 수정할 에셋인지 실행·관리 구조인지 조사 방향을 정합니다.

성능과 제작 비용

GPU만 가벼워져도 CPU가 제한하면 총 프레임은 거의 그대로일 수 있습니다. 가장 긴 막대도 대기를 포함할 수 있어 실제로 제한하는 관계를 확인해야 합니다.

이 예제의 목표

내부 해상도 변화에 Frame과 GPU 시간이 함께 반응하는지 확인합니다.

실습 절차

진단용으로 프레임 제한과 VSync 영향을 확인합니다. 같은 경로의 안정 구간에서 stat unit의 Frame·Game·Draw·GPU를 기록한 뒤 내부 해상도만 낮춰 다시 비교합니다.

예상 결과와 이유

GPU 화면 계산이 제한 원인이면 해상도 감소에 GPU와 Frame이 함께 반응할 수 있습니다. CPU 제한이면 GPU는 줄어도 Frame 변화가 작을 수 있습니다.

결과를 어떻게 해석할까

이는 원인 후보를 좁히는 실험입니다. 가장 긴 수치가 대기 시간을 포함할 수도 있으므로 Unreal Insights에서 Game·Render·RHI·GPU 시간축을 대조해 계산과 기다림을 구분합니다.

이 챕터에서 기억할 것

병목 분석은 가장 복잡한 물체가 아니라 프레임을 제한하는 일을 찾는 과정입니다. 부분 최적화와 전체 개선을 구분합니다.

이해한 뒤 확인할 조건

FPS 대신 ms도 기록합니다. 프레임 제한을 풀어 원인을 진단한 뒤 실제 제품 설정으로 되돌려 효과를 확인합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

26 GPU Profiler와 개선 검증

무엇인가

GPU Profiler는 GPU 작업을 이벤트와 렌더 패스별 시간으로 관찰하는 도구입니다. 프로파일링은 실행 중 어디에 시간이 쓰이는지 측정하고 원인 후보를 좁히는 과정입니다.

렌더링에서 맡는 역할

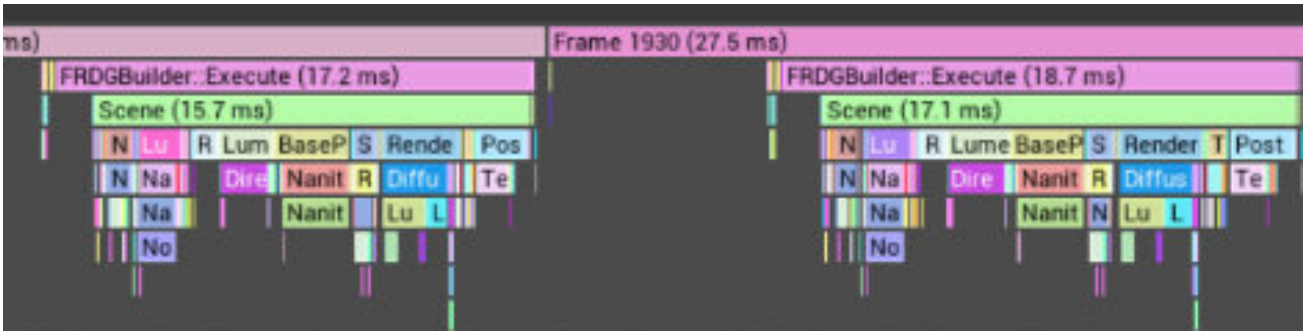
느린 프레임을 구성하는 작업을 찾아 아트 변경의 가설과 연결합니다. 도구는 비용의 위치를 보여 주며, 어떤 제작 선택이 원인인지는 비교 실험으로 검증합니다.

입력과 출력

입력은 캡처한 프레임의 GPU 이벤트와 시간 정보입니다. 출력은 계층·시간축·패스별 비용으로, 다른 조건의 실행과 비교해 효과를 판정합니다.

작동 과정

stat gpu 등으로 큰 비용 범위를 찾고 지원되는 profilegpu·Unreal Insights로 세부를 봅니다. 프레임의 전체 구조 안에서 그림자·표면·반투명·후처리 등을 구분합니다. 부모 이벤트와 세부 구간을 펼쳐 어느 작업이 시간을 쓰는지 찾습니다. 포함 시간과 자체 시간을 구분하고 겹쳐 실행된 작업을 단순 합산하지 않습니다. 작업을 늘리는 에셋·기능에 가설을 세워 한 변수만 바꿉니다. 같은 조건으로 반복 측정하고 해당 패스·전체 프레임·외관이 함께 어떻게 달라졌는지 기록합니다.



외부 실제 도구 화면 · GPU Insights 시간축의 이벤트 계층을 발췌·확대했습니다.

큰 Frame 구간 안에 작은 작업 구간이 포함됩니다. 로그에서 Inclusive는 하위 작업을 포함한 시간, Exclusive는 자체 시간입니다. 부모·자식 시간을 모두 더하지 않습니다. UE 5.6 예시이므로 실제 UI와 기록 항목은 현재 빌드에서 확인합니다.

Tom Looman 문서 수록 · Unreal Engine 도구 화면 · 출처 [60]

원본 일부 발췌·확대 · 색상 유지 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

제작에 연결하고 예제로 확인하기

왜 배경 제작에 필요한가

풀의 그림자 갱신, 큰 반투명, 복잡한 머티리얼 등 제작 변경의 채택 여부를 정할 때 필요합니다. 빠른 한 번의 숫자보다 재현성과 필요한 품질을 유지했는지가 중요합니다.

성능과 제작 비용

캡처·셰이더 준비·로딩 순간은 측정을 흔들 수 있습니다. 안정 구간을 쓰고 대상 기기의 실제 프레임을 확인합니다. 도구의 표시 방식은 UE 버전에 따라 달라집니다.

이 예제의 목표

하나의 가설을 변경·복원하며 패스 감소와 총 프레임 개선을 구분합니다.

실습 절차

CH25에서 GPU 제한을 확인한 구간을 씁니다. 예를 들어 풀의 WPO가 그림자 갱신을 늘린다는 가설을 세우고 WPO만 끕니다. 동일 경로를 워밍업 뒤 세 번씩 기록하고 원래 설정으로 돌려 재현성을 확인합니다.

예상 결과와 이유

가설이 맞으면 그림자 생성 구간이 줄고, GPU 제한이 충분하면 총 시간도 줄 것입니다. 동시에 움직이는 풀과 그림자의 외관 차이가 남을 것입니다.

결과를 어떻게 해석할까

해당 구간만 줄었으면 국소 효과를 확인한 것입니다. 전체 시간이 그대로면 다음 제한 원인을 조사합니다. 변화가 반복되지 않으면 셰이더 준비·로딩·측정 흔들림을 먼저 확인합니다.

이 챕터에서 기억할 것

GPU Profiler는 무거운 일을 찾는 도구이며, 개선은 동일 조건의 변경·복원 실험으로 증명합니다. 패스 감소와 전체 제품 목표 달성을 구분합니다.

이해한 뒤 확인할 조건

이벤트 표시와 캡처 방법은 UE 버전에 따라 다릅니다. stat gpu로 큰 범위를 보고 profilegpu 또는 Unreal Insights로 좁히며, 중첩·동시 실행 시간을 단순 합산하지 않습니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

[29] Beyond Print String · GPU Profiler

종합 실습 원인에서 제작 결정까지

질문을 좁힙니다

골목 전체를 최적화한다는 목표 대신 특정 카메라 구간이 느린 이유를 묻습니다. 이동 경로를 고정하고, CH25의 방법으로 CPU·GPU 중 제한 원인을 먼저 좁힙니다.

예상을 적습니다

GPU의 그림자 생성이 무겁고 풀 주변 페이지가 계속 갱신된다면 WPO가 원인 후보입니다. 바람만 끄면 페이지 갱신과 해당 구간 시간이 줄 것이라고 먼저 적습니다. CH22의 원리를 사용한 가설입니다.

한 번에 하나를 바꿉니다

해상도·조명·카메라는 유지하고 바람만 끕니다. 진단 시각화로 갱신 상태를 보고, 시각화를 끈 뒤 동일 경로를 반복 측정합니다. 변경을 되돌려 결과가 재현되는지도 확인합니다.

이득과 손실을 함께 읽습니다

패스 시간 감소, 총 프레임 변화, 풀과 그림자 움직임의 손실을 적습니다. 시간이 줄어도 필요한 근거리 외관을 잃었다면 그대로 채택하지 않습니다. 먼 구간의 바람만 줄이는 대안을 다시 시험합니다.

다른 장면으로 옮깁니다

배운 것은 바람을 끄라는 규칙이 아닙니다. 변하는 형상이 캐시 재사용을 줄인다는 원리를 알고, 읽히는 움직임과 갱신 비용을 거리별로 배분하는 판단입니다.

용어집

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

CPU와 GPU

CPU는 게임 상태와 작업 준비 등을 처리하는 범용 프로세서, GPU는 많은 그래픽 계산을 병렬로 처리하는 프로세서입니다. [CH01·25]

스레드

프로그램의 명령을 이어 실행하는 흐름입니다. 렌더 스레드가 CPU에서 일한다는 점과 GPU의 실행을 구분합니다. [CH01·25]

렌더 스레드

주로 CPU에서 장면의 가시성과 그릴 명령을 준비하는 실행 흐름입니다. [CH01·25]

RHI와 그래픽 API

RHI는 엔진의 그래픽 명령을 플랫폼별 그래픽 API에 연결하는 계층입니다. API는 기능을 사용하기 위해 정한 요청 방식입니다. [CH01·25]

프레임과 렌더 패스

프레임은 화면 한 장을 만드는 처리 구간이며, 패스는 그 안에서 특정 입력과 결과를 다루는 작업 단위입니다. [CH01·26]

지오메트리와 메시

지오메트리는 형태와 위치에 관한 기하 정보입니다. 이 문서의 메시는 주로 삼각형으로 구성한 3D 표면 형상입니다. [CH01·17]

정점

삼각형의 꼭짓점입니다. 위치 외에 표면 방향과 텍스처 좌표 같은 정보도 가질 수 있습니다. [CH01·10]

좌표 변환과 투영

모델 내부의 위치를 장면·카메라 기준으로 바꾸고 화면 위치에 대응시키는 과정입니다. [CH01]

셰이더

GPU에서 실행되는 계산 프로그램입니다. 정점과 화면의 표면 값 등을 처리하는 여러 종류가 있습니다. [CH01·10]

래스터화

화면에 투영한 삼각형이 어느 화면 위치를 덮는지 판정하는 과정입니다. [CH01]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

샘플과 프래그먼트

래스터 문맥의 샘플은 커버리지를 검사하는 화면 위치입니다. 프래그먼트는 삼각형에서 생긴 화면 위치별 처리 후보입니다. [CH01·05]

보간

알고 있는 값 사이에서 중간 위치의 값을 계산하는 것입니다. 삼각형 내부의 UV 등을 구하는 데 쓰입니다. [CH01]

UV와 텍스처 샘플

UV는 표면을 이미지의 위치에 대응시키는 2D 좌표입니다. 텍스처 샘플은 그 좌표에서 이미지 값을 읽는 작업입니다. [CH01·19]

법선과 노멀 맵

법선은 표면 방향입니다. 노멀 맵은 작은 표면 방향 변화를 이미지로 표현해 빛 반응에 사용합니다. [CH03·07]

탄젠트

표면을 따라 놓인 방향 기준입니다. 탄젠트 공간 노멀 맵은 이 로컬 방향 기준을 이용해 표면 방향 변화를 표현합니다. [CH03]

버퍼와 렌더 타깃

버퍼는 계산 데이터를 보관하는 메모리 공간입니다. 렌더 타깃은 그리기 결과를 기록할 이미지 자원입니다. [CH01·03]

GBuffer와 Base Pass

GBuffer는 조명에 필요한 표면 정보 묶음입니다. Base Pass는 주로 불투명 표면 정보를 준비하는 작업입니다. [CH03]

DBuffer와 데칼

데칼은 다른 표면에 투영하는 얼룩·표식 등의 표현입니다. DBuffer는 Base Pass 전에 그 표면 수정 정보를 준비합니다. [CH04]

깊이 버퍼와 Early Z

깊이 버퍼는 앞쪽 표면 깊이를 저장합니다. Early Z는 가능한 경우 이 정보로 가려진 후보를 일찍 제외합니다. [CH05]

Opaque와 Masked와 Translucent

각각 불투명, 위치별로 남길지 버리는 구멍 마스크, 뒤의 색과 합성하는 반투명 표면을 가리킵니다. [CH06·12]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

Overdraw와 커버리지

Overdraw는 같은 화면 위치의 반복 처리입니다. 커버리지는 도형이 화면의 어느 범위를 덮는지를 뜻합니다. [CH06]

PBR

재료와 빛의 상호작용을 물리적으로 그럴듯하게 근사하는 제작 방식입니다. [CH07]

Base Color와 Metallic과 Roughness

재질의 기본 색, 금속성, 반사가 퍼지는 정도를 제어하는 입력입니다. 최종 화면 색과 역할이 다릅니다. [CH07]

Material Instance와 파라미터

인스턴스는 원본 재질 구조를 공유하는 설정값 변형입니다. 파라미터는 바꿀 수 있도록 노출한 입력값입니다. [CH07·10]

BRDF와 BSDF

BRDF는 방향에 따른 반사 반응을 기술합니다. BSDF는 표면을 통한 투과까지 포괄하는 더 넓은 개념입니다. [CH08]

Fresnel과 F0

Fresnel은 각도에 따른 계면 반사율의 변화입니다. F0는 법선 방향 입사의 반사율입니다. [CH09]

컴파일과 셰이더 순열

컴파일은 실행할 프로그램을 준비하는 과정입니다. 순열은 기능·정적 설정 등의 조합에 따른 실행 버전을 뜻합니다. [CH10]

Static Switch와 WPO

Static Switch는 컴파일할 때 경로를 고릅니다. WPO는 정점의 월드 위치에 변화를 더하는 재질 입력입니다. [CH10]

캐시와 대역폭

캐시는 데이터를 다시 읽기 쉽게 두는 임시 저장 영역입니다. 대역폭은 일정 시간에 옮길 수 있는 데이터 양입니다. [CH03·10]

Slab과 Closure

Slab은 Substrate의 재료 정의 단위입니다. Closure는 실행 시 조명에 대해 평가할 재료 반응의 단위를 가리킵니다. [CH11]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

파라미터 블렌딩

복수 재료의 설정값을 하나의 재료 반응으로 합쳐 근사하는 방식입니다. [CH11]

알파 합성과 굴절

알파 합성은 앞뒤 색을 비중에 따라 섞는 계산입니다. 굴절은 물질 경계에서 빛 방향이 바뀌는 현상입니다. [CH12]

SSS와 MFP

SSS는 표면 아래 빛의 산란입니다. MFP는 내부 상호작용 사이의 평균 거리 스케일을 가리킵니다. [CH13]

Actor와 컴포넌트

Actor는 레벨에 배치하는 오브젝트 단위입니다. 컴포넌트는 그 안의 메시·조명 등 기능 요소입니다. [CH14·15]

Draw Call과 메시 섹션

Draw Call은 GPU에 그리기를 제출하는 명령입니다. 섹션은 보통 재질을 따로 적용할 수 있는 메시의 구역입니다. [CH14]

배칭과 인스턴싱

배칭은 호환되는 작업을 묶는 접근입니다. 인스턴싱은 메시 등의 데이터를 공유해 반복 배치를 처리하는 방식입니다. [CH14·15]

ISM과 HISM

반복 Static Mesh를 컴포넌트 안에서 관리하는 기능입니다. HISM은 공간적으로 묶은 계층을 활용합니다. [CH15]

LOD와 FOV

LOD는 필요한 화면 디테일에 맞춘 대체 표현입니다. FOV는 카메라가 담는 시야각입니다. [CH16]

클러스터와 폴백 메시

Nanite 클러스터는 작은 삼각형 묶음입니다. 폴백 메시는 다른 기능이나 조건에서 사용하는 대체 표현입니다. [CH17]

컬링과 bounds

컬링은 불필요한 가시 후보를 제외하는 과정입니다. bounds는 이런 판정 등에 쓰는 물체를 둘러싼 범위입니다. [CH18]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

밉맵과 필터링

밉맵은 축소 이미지의 계층입니다. 필터링은 필요한 범위의 값을 읽고 섞어 화면에 대응시키는 처리입니다. [CH19]

스트리밍 풀과 I/O

풀은 관리할 메모리 예산의 한 묶음입니다. I/O는 디스크 등에서 데이터를 읽고 옮기는 입출력 작업입니다. [CH20]

Screen Trace와 거리장

Screen Trace는 화면 정보로 경로를 조사합니다. 거리장은 표면까지의 거리를 저장한 장면 표현입니다. [CH21]

Surface Cache와 Cards

Surface Cache는 표면 정보를 빠르게 조회하는 체계입니다. Cards는 표면을 여러 방향에서 담는 표현에 사용됩니다. [CH21]

레이 트레이싱과 가속 구조

광선과 물체의 교차를 추적하는 계산과, 교차 후보를 빨리 찾도록 준비한 자료 구조입니다. [CH21]

VSM 페이지와 캐스터

페이지는 가상 그림자 맵의 작은 영역입니다. 캐스터는 그림자를 만드는 물체입니다. [CH22]

캐시 무효화

예전에 계산해 둔 결과가 현재 장면에 맞지 않아 다시 계산해야 하는 상태입니다. [CH22]

히스토리와 지터

히스토리는 이전 프레임의 저장 정보입니다. 지터는 프레임마다 샘플 위치를 조금씩 바꾸는 동작입니다. [CH23]

모션 벡터와 재투영

모션 벡터는 화면상 이동 정보입니다. 재투영은 과거 정보를 현재 화면 위치에 대응시키는 과정입니다. [CH23]

고스팅

과거 정보가 잘못 남아 움직이는 표면 뒤로 끌리거나 번져 보이는 시간적 아티팩트입니다. [CH23]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

내부 해상도와 업스케일러

내부 해상도는 주요 장면 계산의 크기입니다. 업스케일러는 그 결과 등을 이용해 더 큰 출력 화면을 만드는 기능입니다. [CH24]

병목과 VSync

병목은 처리율을 제한하는 원인입니다. VSync는 디스플레이 갱신에 출력 시점을 맞추는 기능입니다. [CH25]

중앙값과 P95

중앙값은 크기순 가운데 값입니다. P95는 측정값의 95%가 그 이하에 놓이는 경계로, 느린 프레임을 살필 때 사용합니다. [CH25·26]

프로파일링과 워밍업

프로파일링은 실행 중의 비용 측정입니다. 워밍업은 셰이더 준비·로딩 등을 끝내 안정된 비교 조건을 만드는 과정입니다. [CH26]

톤매핑과 Bloom

톤매핑은 장면 밝기를 표시할 범위로 옮기는 처리입니다. Bloom은 밝은 부분이 주변으로 번지는 화면 효과입니다. [CH01]

Scalability와 쿠킹

Scalability는 엔진의 품질 설정 묶음입니다. 쿠킹은 대상 플랫폼에서 실행할 배포용 데이터를 준비하는 과정입니다. [CH26]

심화와 적용 조건

1 스레드와 파이프라인 도식의 범위

RHI는 인터페이스 계층의 이름입니다. RHI 작업이 별도 스레드에서 처리되는 방식과 병렬화 구성은 플랫폼·설정에 따라 달라집니다. 본문의 단계 그림은 작업 의존 관계를 이해하기 위한 것입니다. 실제 GPU 프레임은 그림자·추적·후처리 등의 분기와 동시 실행을 포함하므로 GPU 캡처에서 확인합니다.

2 BRDF 수식을 더 읽을 때

대표적인 미세면 스펙큘러 항은 D×F×G를 4(N·L)(N·V)로 나눈 형태로 설명할 수 있습니다. D는 미세면 방향 분포, F는 Fresnel 반사율, G는 미세면 사이의 가림입니다. N은 표면 법선, L은 빛 방향, V는 시선 방향이며 점 기호는 두 방향의 관계를 계산하는 내적입니다. 실제 UE 재질의 모든 경로가 이 한 식과 같은 구현이라는 뜻은 아닙니다.

3 Substrate 저장 형식과 버전

이 문서는 UE 5.8 문서와 실습 프로젝트의 5.8.4를 기준으로 합니다. Adaptive GBuffer의 지원 플랫폼과 단순화 조건, DBuffer 요구 사항은 해당 버전의 공식 문서에서 확인합니다. 60Hz 지향은 설계 목표이며 실제 게임의 60 FPS 달성을 보장하는 수치가 아닙니다. 프로젝트 범위 변경은 재시작이나 셰이더 재컴파일을 요구할 수 있습니다.

4 GPU 프로파일러의 표시와 시간 합계

UE 5.6 이후 구성에서는 구형 GPU Visualizer 팝업과 같은 표시를 전제로 하지 않고 로그와 Unreal Insights의 GPU 트랙을 확인합니다. 중첩 이벤트, 동시 실행과 CPU·GPU의 대기 때문에 항목의 단순 합이 총 프레임과 다를 수 있습니다. 공식 도구 설명과 대상 빌드의 실제 동작을 함께 확인합니다.

5 셰이더 실행과 프로그램 준비

머티리얼 그래프는 컴파일을 거쳐 실행 프로그램이 됩니다. 정적 스위치 등은 서로 다른 프로그램 조합인 셰이더 순열을 만들 수 있습니다. 실행 시간, 컴파일 대기, 쿠킹·배포 준비는 다른 지표입니다. CH10의 WPO 실험으로 세 지표를 한 번에 검증하지 않습니다.

출처 목록

본문의 공식 링크와 외부 이미지 원문을 함께 보관합니다. 모든 자료가 본문의 측정 증거라는 뜻은 아닙니다.

01 Rendering Settings

02 Render Dependency Graph

03 Introduction to Performance Profiling and Configuration

04 Forward Shading Renderer

05 Programming with Substrate GBuffer Formats

06 Decal Materials

07 Viewport Modes

08 Physically Based Materials

09 Using Fresnel

10 Overview of Substrate Materials

11 Material Editor UI

12 Material Inputs

13 Lit Translucency

14 Using Refraction

15 Subsurface Profile Shading Model

16 Mesh Drawing Pipeline

17 Instanced Static Mesh Component

18 Optimizing LOD Screen Size Per Platform

출처 목록 계속

본문의 공식 링크와 외부 이미지 원문을 함께 보관합니다. 모든 자료가 본문의 측정 증거라는 뜻은 아닙니다.

19 Nanite Virtualized Geometry

20 Texture Asset Editor

21 Texture Streaming Overview

22 Texture Streaming Metrics

23 Lumen Technical Details

24 Lumen Performance Guide

25 Virtual Shadow Maps

26 Anti-Aliasing and Upscaling

27 Temporal Super Resolution

28 Stat Commands

29 Beyond Print String · GPU Profiler

30 Visibility and Occlusion Culling

31 Real Shading in Unreal Engine 4 · Brian Karis

32 Scripting the Editor Using Python

33 Parallel Rendering Overview

34 Direct3D Rasterizer Stage

35 Material Instances

36 Cornellbox pathtracing · Trevorgoodchild · Public domain

출처 목록 계속

본문의 공식 링크와 외부 이미지 원문을 함께 보관합니다. 모든 자료가 본문의 측정 증거라는 뜻은 아닙니다.

37 LearnOpenGL Coordinate Systems · 이미지 출처

38 LearnOpenGL Anti Aliasing · 이미지 출처

39 LearnOpenGL Deferred Shading · 이미지 출처

40 LearnOpenGL PBR Lighting · 이미지 출처

41 LearnOpenGL PBR Theory · 이미지 출처

42 LOD Example · Sirotk · CC BY-SA 3.0

43 Mipmap Aliasing Comparison · BillyBob CornCob · CC0

44 LearnOpenGL 이미지 이용 조건 · CC BY 4.0

45 Epic Nanite Preserve Area 비교 이미지 · UE 5.5

46 Decal Materials 비교 이미지 · UE 5.5

47 LearnOpenGL Depth Testing 비교 이미지

48 Android Developers Materials and Shaders

49 Substrate 코팅 비교 이미지 · UE 5.4

50 Subsurface 비교 이미지 · UE 5.5

51 Mesh Drawing Pipeline 도식 · UE 5.5

52 LearnOpenGL Instancing 렌더 예시

53 Visibility and Occlusion Culling 단계별 그림

54 Texture Streaming Metrics 통계 화면 · UE 5.5

출처 목록 계속

본문의 공식 링크와 외부 이미지 원문을 함께 보관합니다. 모든 자료가 본문의 측정 증거라는 뜻은 아닙니다.

55 Lumen 반사 조명 비교 이미지 · UE 5.5

56 VSM 캐시와 무효화 범위 · UE 5.4

57 Anti Aliasing 전후 이미지 · UE 5.5

58 TSR 동일 출력 비교 · UE 5.5

59 Unreal Insights 실측 화면 · John Farrow

60 GPU Insights와 ProfileGPU 화면 · Tom Looman

61 New GPU Profiler and RHI Submission Pipeline · Epic

62 Graphics Programming Overview · Epic